**GAZİ ÜNİVERSİTESİ**

**TEKNOLOJİ FAKÜLTESİ**

**BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**BMT317 – Algoritmalar**

# Sıralama Algoritmalarının Performans Karşılaştırması

Öğrenci Adı Soyadı: Gülce Duru KOÇ

Öğrenci Numarası: 22181616036

Öğretim Elemanı: Doç. Dr. Adem Tekerek

Tarih: 23 Ekim 2026

# İçindekiler

1. Giriş....4

2. Teorik Bilgi....4

2.1 Karşılaştırma ölçütleri....4

2.2 Algoritmalar ve teorik karmaşıklıklar....5

2.3 Shell Sort ve gap dizileri....5

2.4 Quick Sort ve pivot seçimi....5

2.5 Bellek kullanımı....6

3. Algoritmaların Gerçeklenmesi....6

3.1 Genel yapı....6

3.2 Algoritma ayrıntıları....7

3.3 Sayaç kuralları....7

3.4 Doğruluk kontrolü....8

3.5 Adil karşılaştırma....8

3.6 Proje klasörleri ve modüllerin görevleri....8

3.7 Veri akışı ve ölçüm sınırı....9

3.8 CSV kayıtları ve kontrol noktaları....9

3.9 Otomatik test yaklaşımı....10

4. Deneysel Çalışma....10

4.1 Deney ortamı....10

4.2 Veri üretimi....11

4.3 Deney tasarımı....11

4.4 Bellek ölçümü....12

4.5 Kayıt ve teslim edilen veriler....12

4.6 Sınırlılıklar....13

5. Performans Sonuçları....13

5.1 Doğruluk....13

5.2 Sonuç tabloları....14
5.3 Veri boyutuna göre çalışma süresi....16
5.4 Karşılaştırma sayısı....18
5.5 Swap / veri hareketi....19
5.6 Veri dağılımının performansa etkisi....20
5.7 Bellek kullanımı....24
5.8 Bonus 1 – Quick Sort pivot karşılaştırması....25
5.9 Bonus 2 – Shell Sort gap karşılaştırması....27
5.10 Bonus 3 – n = 500.000....28
5.11 Bonus 4 – Paralel Merge Sort (teorik)....29
5.12 Bonus 5 – Bubble Sort adım görselleştirmesi....30
5.13 Tüm zorunlu deney sonuçları (ortalama, 5 tekrar)....33
6. Karşılaştırma ve Tartışma....38
6.1 O($n^2$) algoritmaların süresi veri boyutu arttıkça nasıl değişti?....38
6.2 O(n log n) algoritmalar neden büyük veri kümelerinde avantajlıdır?....39
6.3 Insertion Sort sıralı verilerde neden başarılıdır?....40
6.4 Quick Sort'un performansı pivot seçimine neden bağlıdır?....40
6.5 Merge Sort veri dağılımından nasıl etkileniyor?....41
6.6 Heap Sort ile Merge Sort büyük veri kümelerinde nasıl karşılaştırılır?....41
6.7 Shell Sort'un performansı gap dizisinden nasıl etkilenir?....42
6.8 Teorik Big-O değerleri deneysel sonuçlarla örtüşüyor mu?....43
6.9 Temel araştırma sorusunun yanıtı....43
7. Sonuç....44
8. Kaynakça....44

# 1. Giriş

Bu projede yedi sıralama algoritmasını aynı veri üretimi ve sayaç tanımlarıyla karşılaştırdım. Çalışmanın odağı, yalnızca bir algoritmanın daha hızlı olduğunu göstermek değil; süre farkının hangi karşılaştırma ve veri hareketlerinden kaynaklandığını incelemektir. Örneğin 100.000 elemanlı rastgele veride Selection Sort'un karşılaştırma sayısı $n(n-1)/2$ olurken Merge Sort diziyi seviyeler hâlinde birleştirir. Bu farkı gerçek CSV kayıtlarındaki süre ve sayaçlarla birlikte değerlendirdim. Bellek ölçümleri süre deneylerinden ayrı yürütüldü.

**Temel araştırma sorusu.** Farklı veri büyüklükleri ve veri dağılımları için yedi algoritmadan hangisi veya hangileri daha avantajlıdır ve elde edilen sonuçlar algoritmaların teorik zaman ve bellek karmaşıklıklarıyla ne ölçüde örtüşmektedir?

**Kapsam.** Zorunlu deney matrisi 6 veri boyutu (100; 1.000; 5.000; 10.000; 50.000; 100.000), 4 veri dağılımı (rastgele, sıralı, ters sıralı, kısmen sıralı) ve 7 algoritmadan oluşan 168 kombinasyondur. Her kombinasyon bir ısınma (warm-up) ve beş gerçek ölçümle çalıştırılmış, toplam **840 ölçüm** yapılmıştır; tüm ölçümlerde sonuç dizisi doğrulanmıştır. Ayrıca bellek analizi ve ödevin beş bonus çalışması yapılmıştır: Quick Sort pivot karşılaştırması, Shell Sort gap karşılaştırması, n = 500.000 deneyi, paralel Merge Sort'un teorik incelemesi ve Bubble Sort adım görselleştirmesi.

**Raporun yapısı.** Bölüm 2 teorik bilgiyi, Bölüm 3 gerçeklemeleri, Bölüm 4 deney tasarımını ve ortamı, Bölüm 5 ölçüm sonuçlarını sunar. Bölüm 6 sonuçları teoriyle karşılaştırır ve ödevde sorulan soruları yanıtlar; Bölüm 7 genel sonuçları verir.

# 2. Teorik Bilgi

## 2.1 Karşılaştırma ölçütleri

Bir sıralama algoritmasının maliyeti genellikle üç nicelikle anlatılır: veri karşılaştırma sayısı, veri hareketi (atama ya da yer değiştirme) sayısı ve ek bellek gereksinimi. Karşılaştırma tabanlı sıralamada en kötü durumda en az $\Omega(n \log n)$ karşılaştırma gerektiği bilinir; bu yüzden $n \log n$ sınıfı bu problem için asimptotik olarak en iyi sınıftır. Big-O gösterimi sabit çarpanları ve düşük mertebeden terimleri gizler; bu çalışma, gizlenen bu ayrıntıların gerçek süreye etkisini de gözlemlemeyi amaçlar (Cormen ve diğerleri, 2022, sıralama bölümleri).

## 2.2 Algoritmalar ve teorik karmaşıklıklar

**Tablo 1. Algoritmaların teorik karmaşıklıkları**

| Algoritma | En iyi | Ortalama | En kötü | Ek bellek | Kararlı |
|---|---|---|---|---|---|
| **Selection** | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | O(1) | Hayır |
| **Insertion** | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) | Evet |
| **Bubble (erken durmalı)** | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) | Evet |
| **Shell (klasik)** | O(n log n) | Gap/veriye bağlı | $O(n^2)$ | O(log n)* | Hayır |
| **Quick** | O(n log n) | O(n log n) | $O(n^2)$ | O(log n) | Hayır |
| **Merge** | O(n log n) | O(n log n) | O(n log n) | O(n) | Evet |
| **Heap** | O(n)* | O(n log n) | O(n log n) | O(1) | Hayır |

** Bu çalışmadaki Shell Sort gerçeklemesi gap listesini bir listede sakladığı için O(log n) ek alan kullanır. Quick Sort'un O(log n) değeri, küçük parçaya özyineleme yapan gerçeklemeye aittir; bu önlem alınmasa özyineleme derinliği en kötü durumda O(n) olurdu.*

*Heap için O(n) en iyi durum tüm değerler eşit olduğunda mümkündür: aşağı süzme ilk seviyede durur. Deney verileri farklı 0..n−1 değerleri içerdiğinden bu özel durum ana deneylerde yer almaz. Teorik tabloda Quick'in ortalaması rastgele girdi varsayımına dayanır.*

## 2.3 Shell Sort ve gap dizileri

Projede klasik gap listesi önce n//2, ardından gap//2 hesaplanarak oluşturulur. n=10 için [5, 2, 1], n=100 için [50, 25, 12, 6, 3, 1] elde edilir; dolayısıyla değerler her zaman ikinin kuvvetleri değildir. Her gap için kaydırmalı Insertion Sort uygulanır. Bu klasik yöntemin genel en kötü durum üst sınırı $O(n^2)$'dir (Knuth, 1998, Bölüm 5.2.1). Ölçümlerde daha yavaş bir büyüme görülmesi bu sınırı değiştirmez. Bonus yönteminde h=3h+1 ile üretilen ve n'den küçük olan Knuth aralıkları ters sırayla kullanılır. Knuth dizisinin klasik incelemesi $O(n\sqrt{n})$ en kötü durum sınırı verir (Sedgewick ve Wayne, 2011; Shell sınıfı belgeleri). Bizim kodumuz her iki yöntemde de gap listesini sakladığından yardımcı alan O(log n)'dir.

## 2.4 Quick Sort ve pivot seçimi

Quick Sort'ta orta pivot, alt dizinin (başlangıç+bitiş)//2 indeksindeki değerdir; medyan hesaplanmaz. İki uçtan ilerleyen indeksler pivotun yanlış tarafında kalan değerleri değiştirir. Dengeli bölünmelerde bölümleme ağacının yüksekliği O(log n), toplam iş O(n log n) olur.

Tekrarlanan dengesiz bölünmelerde ağaç yüksekliği O(n), süre $O(n^2)$ olabilir (Hoare, 1962; Cormen ve diğerleri, 2022). Bu ağacın yüksekliği, projedeki Python çağrı yığınıyla aynı değildir. _araligi_sirala küçük parçayı özyinelemeye verir, büyük parçanın sınırlarını değiştirip döngüye devam eder. İç içe her çağrı en fazla yarı büyüklükte bir parça aldığı için gerçek çağrı yığını O(log n) ile sınırlıdır; bu önlem karesel karşılaştırma maliyetini ortadan kaldırmaz.

### 2.5 Bellek kullanımı

Merge Sort birleştirme için n elemanlı yardımcı dizi gerektirir (O(n)). Quick Sort yerinde çalışır ancak özyineleme yığını kullanır; en kötü durumda O(n) olabilecek yığın derinliği, önce küçük parçanın özyinelemeyle sıralanmasıyla O(log n) ile sınırlanır. Heap Sort tamamen yerinde çalışır ve yinelemeli süzme kullanıldığında O(1) ek bellek yeterlidir. Selection, Insertion ve Bubble Sort da O(1) ek bellek kullanır.

## 3. Algoritmaların Gerçeklenmesi

### 3.1 Genel yapı

Yedi algoritma Python ile, hazır sort() veya sorted() çağrılarına dayanmadan, algoritmalar/ klasöründe ayrı modüller olarak yazılmıştır. Her fonksiyon fonksiyon(dizi, sayaclar) biçiminde çağrılır, verilen listeyi yerinde sıralar ve karşılaştırma/hareket sayılarını Sayaclar nesnesine ekler. Algoritmalar deney, kayıt ve arayüz kodundan bağımsızdır; böylece aynı kod hem testlerde hem ölçümlerde kullanılır.

#### 3.1.1 Neden Python?

Bu çalışmada amaç, sıralama algoritmalarının adımlarını açık biçimde göstermek ve aynı ölçüm kuralları altında karşılaştırmaktır. Python'un okunabilir sözdizimi; karşılaştırma, kaydırma ve takas işlemlerini kod üzerinde izlemeyi kolaylaştırır. Algoritmaları aynı liste yapısı ve ortak Sayaclar nesnesiyle uygulamak, işlem sayılarının tutarlı tanımlanmasını ve doğruluk testlerinin ortak bir yapıda yürütülmesini sağlar.

Standart kütüphanedeki unittest, random.Random, perf_counter_ns, csv ve tracemalloc araçları test, tekrar üretilebilir veri, süre ölçümü, kayıt ve bellek analizi için yeterlidir. Bu nedenle algoritma ve deney altyapısı ek paket gerektirmez. Buna karşılık ölçülen süreler Python yorumlayıcısının ve sayaç artırımlarının maliyetini de içerir. Sonuçları diller arasında

bir hız karşılaştırması olarak değil, bu projedeki Python gerçeklemelerinin karşılaştırması olarak değerlendiriyorum.

### 3.2 Algoritma ayrıntıları

- **Selection Sort:** Sıralanmamış bölümün en küçük elemanını bulur ve başa yerleştirir. Konumu zaten doğruysa yer değiştirme yapmaz; bu nedenle hareket sayısı çok düşüktür.

- **Insertion Sort:** Her elemanı geçiciye alıp kendisinden büyükleri sağa kaydırarak yerine yerleştirir. Başarısız son karşılaştırma da sayılır.

- **Bubble Sort:** Komşu elemanları karşılaştırıp takas eder. İki iyileştirme içerir: her turda sona yerleşen eleman dışarıda bırakılır ve takas olmayan turda algoritma erken durur.

- **Shell Sort:** Her aralıkta kaydırmalı eklemeli sıralama yapar. Varsayılan aralıklar n//2, ardından gap//2 ile üretilir; son aralık 1 (klasik); bonus karşılaştırmada Knuth dizisi kullanılır.

- **Quick Sort:** İki yönlü (Hoare tipi) bölümleme kullanır. Pivot varsayılan olarak orta indeksteki elemandır; bonus için ilk eleman seçeneği vardır. Küçük parça özyinelemeyle, büyük parça döngüyle işlenir.

- **Merge Sort:** Yukarıdan aşağıya (top-down) bölme uygular. Alt diziler oluşturmaz; tüm çağrılar n elemanlı tek bir yardımcı tamponu paylaşır. Eşitlikte soldan seçerek kararlılığı korur.

- **Heap Sort:** Max-heap kurar, kökü sona taşıyıp yığını küçülterek yinelemeli (özyinelemesiz) aşağı süzme uygular.

### 3.3 Sayaç kuralları

Sayaçlar, algoritmanın veri üzerinde yaptığı işlemleri şu kurallarla sayar:

- **Karşılaştırma:** İki veri elemanının her karşılaştırması, başarısız son karşılaştırmalar dahil, 1 sayılır. İndeks ve döngü koşulları sayılmaz.

- **Veri hareketi:** Veri atamaları sayılır. Geçiciye eleman alma ve her dizi yazımı 1 harekettir; dolayısıyla geçici değişkenli bir swap **3 hareket** eder. Merge Sort'ta tampona ve ana diziye yazımlar, Quick Sort'ta pivot kopyası sayılır. İndeks atamaları sayılmaz.

Hareket sayısı bu nedenle “swap sayısı”ndan farklıdır: Selection ve Heap gibi swap yapan algoritmalarda hareket = 3 × swap'tır, Insertion ve Shell gibi kaydıran algoritmalarda ise her kaydırma 1 harekettir. Algoritmalar arası karşılaştırma bu ortak ölçüt üzerinden yapılmıştır.

### 3.4 Doğruluk kontrolü

Her ölçümde projedeki isSorted(dizi) fonksiyonu çalıştırılmış ve ayrıca eleman bütünlüğü (çıktının girdiyle aynı elemanlardan oluşması) frekans farklarıyla denetlenmiştir; yalnızca ikisini birden geçen ölçüm başarılı sayılmıştır. Otomatik testler algoritmaları, veri üretimini, deney altyapısını ve doğrulamayı kapsar. Zorunlu deneylerin 840 ölçümünün hepsi başarılıdır.

### 3.5 Adil karşılaştırma

Her ölçüm başlangıç dizisinin ayrı bir kopyası üzerinde çalışır; bir algoritmanın sıraladığı dizi başka bir algoritmaya verilmez. Sayaç nesnesi her ölçümde yenidir.

### 3.6 Proje klasörleri ve modüllerin görevleri

Aşağıdaki ağaç, rapor hazırlanırken projede bulunan dosyaların görev odaklı özetidir. Paketlerin __init__.py dosyaları ve sonuç klasörlerindeki tekil kayıtlar okunabilirlik için gösterilmemiştir.

SiralamaKarsilastirma/
main.py
algoritmalar/ selection.py, insertion.py, bubble.py, shell.py, quick.py, merge.py, heap.py
cekirdek/ sayaclar.py, dogrulama.py, veri_uretimi.py
deneyler/ olcum.py, deney_yoneticisi.py, kayit.py, tam_deney.py
on_analiz.py, bellek_analizi.py, bonus_pivot.py, bonus_gap.py, bonus_buyuk_veri.py
analizler/ paralel_merge_analizi.py
gorsellestirme/ bubble_gorsel.py, grafikler.py
testler/
sonuclar/ deneyler/, bonuslar/, analizler/, grafikler/
README.md, requirements.txt, .gitignore

main.py küçük örnek çalıştırmayı sunar. Algoritma dosyaları yalnızca verilen dizi ve sayaçla çalışır. Çekirdek paketindeki veri üretimi yerel random.Random nesnesi kullanır; isSorted komşu değerlerin sırasını denetler. Sayaclar veri sınıfının iki alanı sıfırdan başlar ve ölçümler arasında ortak, değişebilir bir global sayaç tutulmaz.

Deney paketinde olcum.py tek algoritma çağrısının başlangıç ve bitiş zamanlarını ölçer; deney_yoneticisi.py warm-up, tekrar ve seed düzenini kurar. kayit.py CSV alanlarını ve sayı biçimlerini tanımlar. tam_deney.py seçilen grupları çalıştırır ve kontrol noktalarını yönetir.

Bonus ölçümleri ayrı dosyalara yazılır. Bellek analizi süre deneylerinden ayrıdır; paralel Merge modülü ise gerçek paralel sıralama çalıştırmadan model hesabı yapar. Grafikler kayıtlı sonuçlardan hazırlanır.

### 3.7 Veri akışı ve ölçüm sınırı

Veri akışı şu sırayı izler: veriUret → başlangıç dizisi → tekOlcum içindeki bağımsız kopya ve Sayaclar → algoritma → sıralılık ve eleman bütünlüğü → ham kayıt → beş tekrarın özeti → CSV. Veri üretici 0 ile n−1 arasındaki aynı değerleri kullanır. Kısmen sıralı veride seçilen konumlar döngüsel kaydırılır; dolayısıyla veri türleri arasında eleman kümesi değişmez.

Aşağıdaki kısa parça deneyler/olcum.py içindeki gerçek ölçüm sınırını gösterir. Kopya ve sayaç saat başlamadan hazırlanır; doğrulama bitiş zamanından sonra yapılır.

```
dizi = baslangic_dizisi.copy()
sayaclar = Sayaclar()


baslangic = time.perf_counter_ns()
algoritma(dizi, sayaclar)
bitis = time.perf_counter_ns()
```

Eleman bütünlüğü için başlangıç dizisinin Counter frekanslarından çıktı frekansları çıkarılır. Bütün farkların sıfır olması ve isSorted sonucunun True olması gerekir. Böylece yanlışlıkla eleman silen fakat sıralı görünen bir çıktı başarılı kabul edilmez. Gerçek tekrarlar 317–321 seed değerlerini kullanır; warm-up ortalamaya alınmaz. Deney yöneticisinin çoklu algoritma çağrısında algoritma sırası tekrar indeksine göre döndürülür. Aşamalı zorunlu çalıştırıcı grupları tek algoritmayla çağırdığı için bu çağrılarda algoritmalar arası sıra dönüşümü oluşmaz.

### 3.8 CSV kayıtları ve kontrol noktaları

Ham CSV her tekrarın seed, süre, karşılaştırma, hareket ve doğrulama sonuçlarını saklar. Özet CSV, aynı algoritma, dağılım ve boyut için beş tekrarın ortalamasını verir. Ham kayıt sayıları ve özet ortalamaları birbiriyle karşılaştırılarak denetlenir.

Tamamlanan deney grupları kontrol noktalarına kaydedilir. Yeniden başlatıldığında beş başarılı tekrarı bulunan gruplar atlanır; eksik gruplar tamamlanmış sayılmaz. Geçici dosya üzerinden kayıt, kesinti sırasında tamamlanmış grupların korunmasını sağlar. Altyapı gerçek bir ölçüm timeout mekanizması içermez; süre ölçümü işlemi durdurmaz.

### 3.9 Otomatik test yaklaşımı

unittest testleri boş ve tek elemanlı girdilerden negatif ve tekrar eden değerlere kadar sıralama doğruluğunu sınar. Küçük örneklerde karşılaştırma ve hareket sayıları elle beklenen değerlerle karşılaştırılır; Merge için kararlılık, Quick için büyük sıralı girdiler ve geçersiz stratejiler ayrıca ele alınır. Veri üretiminde seed tekrar üretilebilirliği ve dağılımlar arasındaki eleman bütünlüğü denetlenir.

Deney altyapısı testlerinde geçici klasörler ve sahte ölçümler kullanılarak CSV başlıkları, tekrar sayıları, kesinti sonrası devam ve mükerrer kayıt önleme kontrol edilir. Bu testler uzun performans deneylerinin yerine geçmez. Tkinter testleri pencere açmadan adım kayıtlarını ve gerçek Bubble Sort ile aynı nihai sonucu denetler; bu, pencerenin masaüstünde görünmesini doğrulayan bir test değildir.

## 4. Deneysel Çalışma

### 4.1 Deney ortamı

**Tablo 2. Deney ortamı**

| **Bileşen** | **Değer** |
|---|---|
| **İşlemci** | Apple M5 (10 çekirdek) |
| **RAM** | 16 GB |
| **İşletim sistemi** | macOS 26.6 |
| **Programlama dili ve sürümü** | Python 3.13.13 (CPython) |
| **Derleyici / çalışma ortamı** | CPython bayt kodu yorumlayıcısı; sanal ortam (.venv) |
| **IDE** | PyCharm |
| **Zamanlayıcı** | time.perf_counter_ns() (Python Software Foundation, time belgeleri) |
| **Bellek ölçümü** | tracemalloc |
| **Grafik kütüphanesi** | matplotlib |
| **Shell Sort gap yöntemi** | Klasik n//2, ardından gap//2 (ana deney); Knuth 1, 4, 13, … (bonus) |
| **Quick Sort pivot yöntemi** | Orta indeksteki eleman (ana deney); ilk eleman (bonus) |

Ana deneylerde Shell için klasik gap ve Quick için orta indeks pivotu kullanılır. Knuth dizisi bonus deneyde gap seçiminin etkisini ölçmek için kullanılmıştır. Orta indeks pivotu sıralı ve

ters sıralı girdilerde dengeli bölünme sağlar. İlk pivot bu girdilerde dengesiz bölünmenin etkisini incelemek için bonusta kullanılmıştır.

Tablodaki işlemci, RAM, çekirdek sayısı ve işletim sistemi 23.10.2026 tarihli yerel sistem okumasıyla doğrulanmıştır. Önceki CSV'lerde deney anına ait donanım ve işletim sistemi metaverisi bulunmadığından aynı yapılandırmanın her oturumda geçerli olduğu bağımsız olarak kanıtlanamaz. IDE bilgisi proje ortamında belirtilen PyCharm'dır; sürümü kaydedilmemiştir.

### 4.2 Veri üretimi

Tüm diziler 0'dan n−1'e kadar farklı tam sayılardan oluşan permütasyonlardır; dört dağılım aynı değer kümesini içerir. Yerel bir random.Random üreteci kullanıldığından üretim deterministiktir ve global rastgele durum etkilenmez.

- **Rastgele:** Fisher–Yates karıştırması.

- **Sıralı:** 0, 1, …, n−1.

- **Ters sıralı:** n−1, n−2, …, 0.

- **Kısmen sıralı:** Sıralı diziden rastgele seçilen yaklaşık %10 konum (en az 2) kendi aralarında döngüsel kaydırılır; geri kalan %90 yerinde kalır.

### 4.3 Deney tasarımı

Her (boyut, dağılım, algoritma) kombinasyonu bir ısınma ölçümü ve beş gerçek ölçümle çalıştırılmış, özet değerler beş ölçümün ortalamasıdır. Isınma ölçümü ilk çalıştırmanın başlangıç maliyetlerini azaltmak amacıyla yapılır; bu etkilerin giderildiği ayrıca ölçülmemiştir. Ortalamaya katılmaz. Isınmanın seed'i 316, gerçek tekrarların seed'leri 317–321'dir; böylece her tekrar farklı rastgele veri kullanır ve her algoritma aynı seed için aynı veriyi alır. Sıralı ve ters sıralı dağılımlar seed'den bağımsızdır, bu yüzden bu dağılımlarda beş tekrar aynı veriyle yapılır. Toplam ölçüm sayısı $6 \times 4 \times 7 \times 5 = 840$'tır.

Süre yalnızca algoritma çağrısını kapsar; veri üretimi, kopyalama, sayaç oluşturma, doğrulama ve kayıt süre dışındadır. Sayaç artırımları algoritmanın parçasıdır ve süreye dahildir.

### 4.4 Bellek ölçümü

Ek bellek tracemalloc ile algoritma çağrısı sırasında oluşan tepe Python tahsisi olarak ölçülmüştür. Süre ve bellek ölçümleri ayrı çalıştırılmıştır, çünkü tracemalloc yorumlayıcıyı yavaşlatır. Bu ölçüm toplam RAM kullanımı ya da matematiksel yardımcı alanın kesin karşılığı değildir; izlenebilen Python tahsislerini gösterir (Python Software Foundation, tracemalloc belgeleri).

### 4.5 Kayıt ve teslim edilen veriler

Her ham ölçüm algoritma, veri tipi, boyut, seed, tekrar numarası, süre (ns ve ms), karşılaştırma sayısı, hareket sayısı ve doğrulama sonuçlarıyla sonuclar/deneyler/ham_<n>.csv dosyalarına; beş tekrarın ortalaması ozet_<n>.csv dosyalarına yazılmıştır. Bonus ve bellek sonuçları sonuclar/bonuslar/ ve sonuclar/bellek_sonuclari.csv içindedir. Raporun tablo ve grafikleri bu dosyalardan üretilmiştir (gorsellestirme/grafikler.py). Deneyleri yeniden çalıştırma adımları README dosyasındadır.

#### 4.5.1 CSV kayıtlarından örnek kesitler

Şekil 1–3, kayıt düzenini göstermek için kaynak CSV'lerden seçilmiş üçer satır ve ilgili sütunları içerir. Dosyaların tamamı gösterilmemiştir. Sütun başlıkları okunabilirlik için kısaltılmış; hücre değerleri kaynak dosyalardan değiştirilmeden alınmıştır.

**ham_100000.csv — seçilmiş satır ve sütunlar**
n=100000; rastgele; ilk üç tekrar

| Algoritma | Seed | Tekrar | Süre (ms) | Karşılaştırma | Hareket |
|---|---|---|---|---|---|
| quick | 317 | 1 | 170.699625000 | 2191762 | 1251137 |
| quick | 318 | 2 | 171.699333000 | 2229181 | 1253096 |
| quick | 319 | 3 | 177.873750000 | 2310410 | 1237479 |

*Şekil 1. ham_100000.csv kesiti: ham sonuçlar her tekrarı ayrı seed, süre ve sayaç değerleriyle kaydeder.*

**ozet_100000.csv — seçilmiş satır ve sütunlar**
n=100000; rastgele; üç algoritmanın özeti

| Algoritma | Tekrar | Ort. süre (ms) | Ort. karşılaştırma | Ort. hareket |
|---|---|---|---|---|
| shell | 5 | 448.967791800 | 4335791.600000000 | 5886484.400000000 |
| quick | 5 | 174.776333200 | 2238728.600000000 | 1246496.400000000 |
| merge | 5 | 252.013849800 | 1536353.200000000 | 3337856.000000000 |

*Şekil 2. ozet_100000.csv kesiti: özet sonuçlar beş gerçek tekrarın ortalama süre, karşılaştırma ve hareket değerlerini verir.*

**bellek_sonuclari.csv — seçilmiş satır ve sütunlar**
n=10000; rastgele; ayrı tracemalloc ölçümleri

| Algoritma | Boyut | Tepe bellek (bayt) | Tepe bellek (KiB) | Başarılı |
|---|---|---|---|---|
| quick | 10000 | 896 | 0.875000 | True |
| merge | 10000 | 80740 | 78.847656 | True |
| heap | 10000 | 324 | 0.316406 | True |

*Şekil 3. bellek_sonuclari.csv kesiti: ayrı tracemalloc kayıtları bellek analizini destekler; süre deneylerinden bağımsızdır.*

### 4.6 Sınırlılıklar

- CPU sıcaklığı, frekans değişimi ve arka plan yükü deney sırasında kaydedilmedi. Isınma veya işlemci hızının düşmesi olası etkenlerdir; herhangi bir süre farkının kesin nedeni olarak gösterilemez.

- Algoritma süreleri Python yorumlayıcısı, nesne yönetimi ve sayaç artırımı maliyetlerini birlikte içerir. C/C++ ile karşılaştırmalı deney yapılmadığından bu dillere göre hız oranı verilemez. Buradaki süreler incelenen Python gerçeklemelerine aittir.

- Süreler donanıma, işlemci zamanlamasına ve oturum koşullarına bağlıdır; deneyler farklı oturumlarda alındığından mutlak süreler evrensel bir performans garantisi değildir.

- $O(n^2)$ algoritmalar n = 100.000'de dakikalar sürmüştür (Bubble Sort rastgele veride yaklaşık 8,2 dk); bu ölçümler zaman aşımı olmadan tamamlanmıştır. n = 500.000 deneyine bu nedenle yalnızca önceki ölçümlerde daha kısa süren algoritmalar (Shell, Quick, Merge, Heap) alınmıştır.

- Bellek ölçümü yalnızca n = 1.000 ve n = 10.000 için yapılmıştır; n = 10.000'de yalnızca Shell, Quick, Merge ve Heap ölçülmüştür. Pivot ve gap bonusları $n \leq 10.000$ ile sınırlıdır; ilk eleman pivotu sıralı veride $O(n^2)$'ye düştüğü için daha büyük boyut denenmemiştir.

- Her ölçüm grubunda yalnızca beş tekrar vardır; bu, değişkenlik hakkında sınırlı bilgi verir; istatistiksel anlamlılık testi uygulanmamıştır.

## 5. Performans Sonuçları

### 5.1 Doğruluk

Zorunlu deneylerin 840 ölçümünün tamamı isSorted ve eleman bütünlüğü denetimini geçmiştir; 168 özet kombinasyonunun tamamı başarılıdır. Bellek analizindeki 44 ölçüm,

Quick Sort pivot bonusundaki 160, Shell gap bonusundaki 160 ve n = 500.000 deneyindeki 80 ölçüm de başarılıdır.

### 5.2 Sonuç tabloları

Tüm zorunlu sonuçlar Bölüm 5.13'teki 168 satırlık tabloda verilmiştir. Aşağıda n = 1.000 ve n = 100.000 için ortalama sonuçlar yer alır; süreler milisaniyedir.

**Tablo 3. n = 1.000 için ortalama sonuçlar (5 tekrar)**

| Algoritma | Veri tipi | Süre (ms) | Karşılaştırma | Hareket |
|---|---|---|---|---|
| Selection | Rastgele | 21,27 | 499.500 | 2.977 |
| Insertion | Rastgele | 20,88 | 252.946 | 253.953 |
| Bubble | Rastgele | 34,71 | 498.789 | 755.865 |
| Shell | Rastgele | 2,458 | 15.125 | 23.651 |
| Quick | Rastgele | 1,964 | 13.295 | 7.823 |
| Merge | Rastgele | 3,225 | 8.698 | 19.952 |
| Heap | Rastgele | 3,799 | 16.834 | 27.217 |
| Selection | Sıralı | 20,33 | 499.500 | 0 |
| Insertion | Sıralı | 0,160 | 999 | 1.998 |
| Bubble | Sıralı | 0,100 | 999 | 0 |
| Shell | Sıralı | 2,518 | 8.006 | 16.012 |
| Quick | Sıralı | 1,662 | 9.009 | 511 |
| Merge | Sıralı | 3,441 | 4.932 | 19.952 |
| Heap | Sıralı | 3,936 | 17.583 | 29.124 |
| Selection | Ters sıralı | 21,58 | 499.500 | 1.500 |
| Insertion | Ters sıralı | 41,70 | 499.500 | 501.498 |
| Bubble | Ters sıralı | 46,20 | 499.500 | 1.498.500 |
| Shell | Ters sıralı | 1,414 | 11.716 | 20.712 |
| Quick | Ters sıralı | 1,231 | 9.016 | 2.011 |
| Merge | Ters sıralı | 3,429 | 5.044 | 19.952 |
| Heap | Ters sıralı | 3,612 | 15.965 | 24.948 |
| Selection | Kısmen sıralı | 21,35 | 499.500 | 297 |
| Insertion | Kısmen sıralı | 3,748 | 32.654 | 33.653 |
| Bubble | Kısmen sıralı | 23,77 | 490.548 | 94.965 |
| Shell | Kısmen sıralı | 2,159 | 12.273 | 20.351 |

| Algoritma | Veri tipi | Süre (ms) | Karşılaştırma | Hareket |
|---|---|---|---|---|
| Quick | Kısmen sıralı | 1,587 | 9.206 | 1.025 |
| Merge | Kısmen sıralı | 3,743 | 7.567 | 19.952 |
| Heap | Kısmen sıralı | 4,039 | 17.485 | 28.958 |

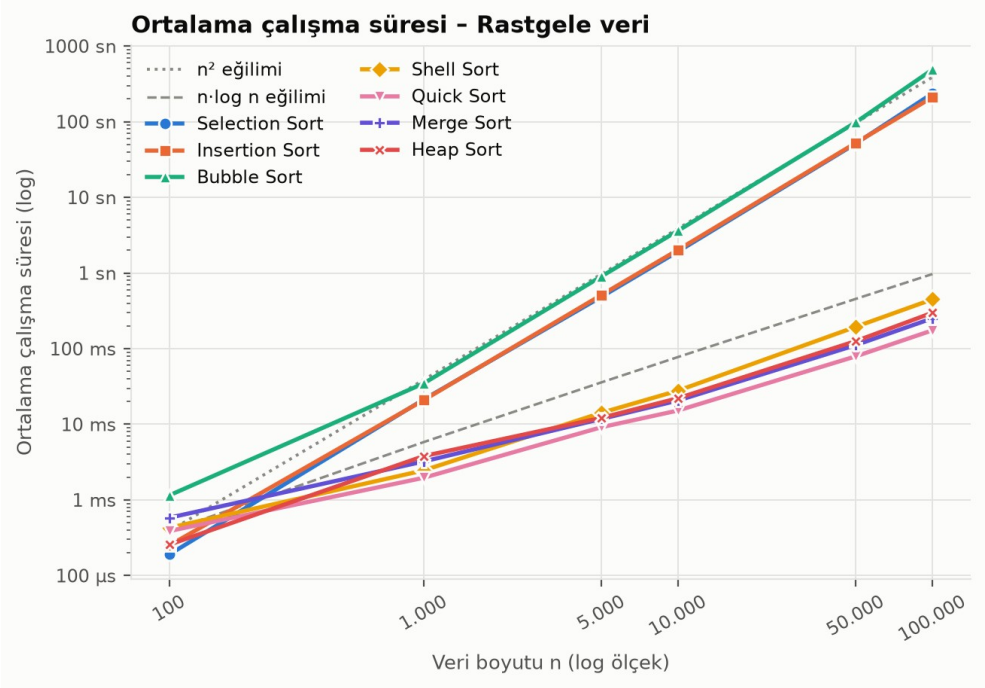
**Tablo 4. n = 100.000 için ortalama sonuçlar (5 tekrar)**

| Algoritma | Veri tipi | Süre (ms) | Karşılaştırma | Hareket |
|---|---|---|---|---|
| Selection | Rastgele | 236.668,8 | 4.999.950.000 | 299.966 |
| Insertion | Rastgele | 210.804,3 | 2.505.370.820 | 2.505.470.830 |
| Bubble | Rastgele | 489.305,1 | 4.999.914.055 | 7.515.812.496 |
| Shell | Rastgele | 448,97 | 4.335.792 | 5.886.484 |
| Quick | Rastgele | 174,78 | 2.238.729 | 1.246.496 |
| Merge | Rastgele | 252,01 | 1.536.353 | 3.337.856 |
| Heap | Rastgele | 298,59 | 3.019.566 | 4.724.852 |
| Selection | Sıralı | 207.822,5 | 4.999.950.000 | 0 |
| Insertion | Sıralı | 11,61 | 99.999 | 199.998 |
| Bubble | Sıralı | 6,896 | 99.999 | 0 |
| Shell | Sıralı | 166,44 | 1.500.006 | 3.000.012 |
| Quick | Sıralı | 93,31 | 1.600.016 | 65.535 |
| Merge | Sıralı | 204,04 | 815.024 | 3.337.856 |
| Heap | Sıralı | 283,53 | 3.112.517 | 4.952.562 |
| Selection | Ters sıralı | 228.004,4 | 4.999.950.000 | 150.000 |
| Insertion | Ters sıralı | 467.263,2 | 4.999.950.000 | 5.000.149.998 |
| Bubble | Ters sıralı | 524.991,0 | 4.999.950.000 | 14.999.850.000 |
| Shell | Ters sıralı | 252,95 | 2.244.585 | 3.844.572 |
| Quick | Ters sıralı | 95,33 | 1.600.030 | 215.535 |
| Merge | Ters sıralı | 206,60 | 853.904 | 3.337.856 |
| Heap | Ters sıralı | 269,05 | 2.926.640 | 4.492.302 |
| Selection | Kısmen sıralı | 229.325,0 | 4.999.950.000 | 29.997 |
| Insertion | Kısmen sıralı | 29.065,9 | 325.263.665 | 325.363.664 |
| Bubble | Kısmen sıralı | 273.544,0 | 4.999.578.750 | 975.490.999 |
| Shell | Kısmen sıralı | 359,63 | 3.465.588 | 4.972.731 |
| Quick | Kısmen sıralı | 102,02 | 1.626.512 | 159.214 |

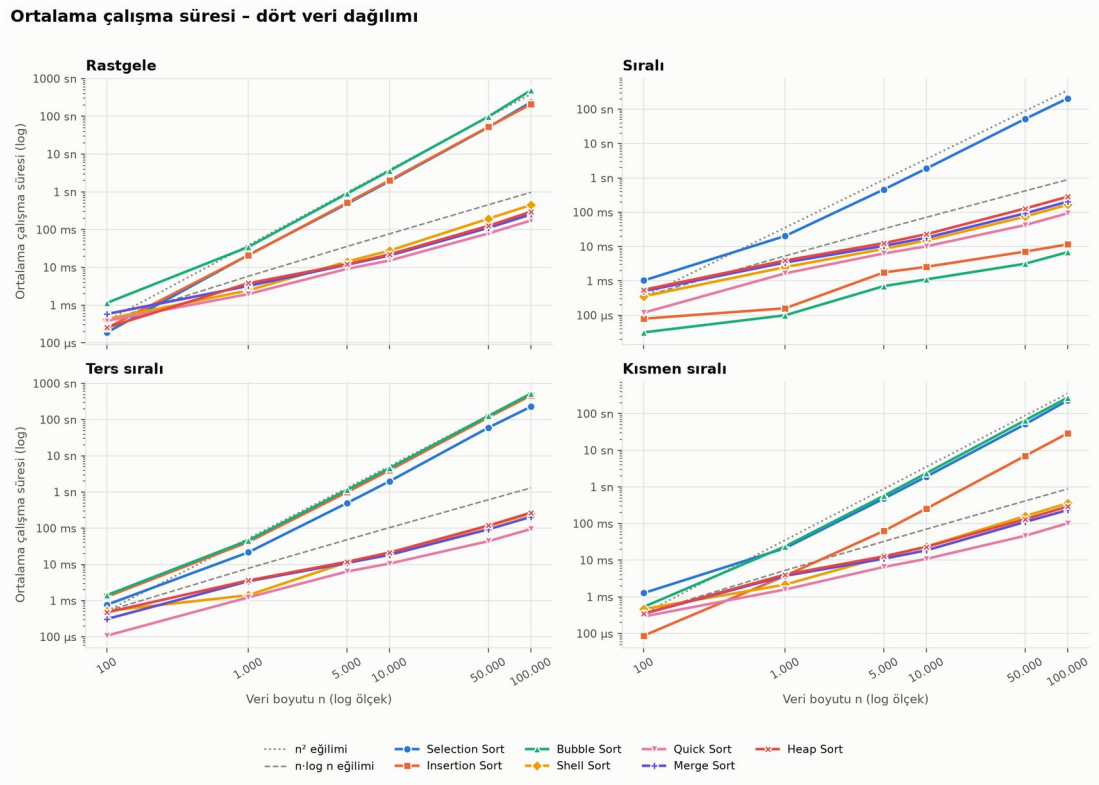
| Algoritma | Veri tipi | Süre (ms) | Karşılaştırma | Hareket |
|---|---|---|---|---|
| Merge | Kısmen sıralı | 234,31 | 1.419.708 | 3.337.856 |
| Heap | Kısmen sıralı | 292,50 | 3.103.405 | 4.926.951 |

### 5.3 Veri boyutuna göre çalışma süresi

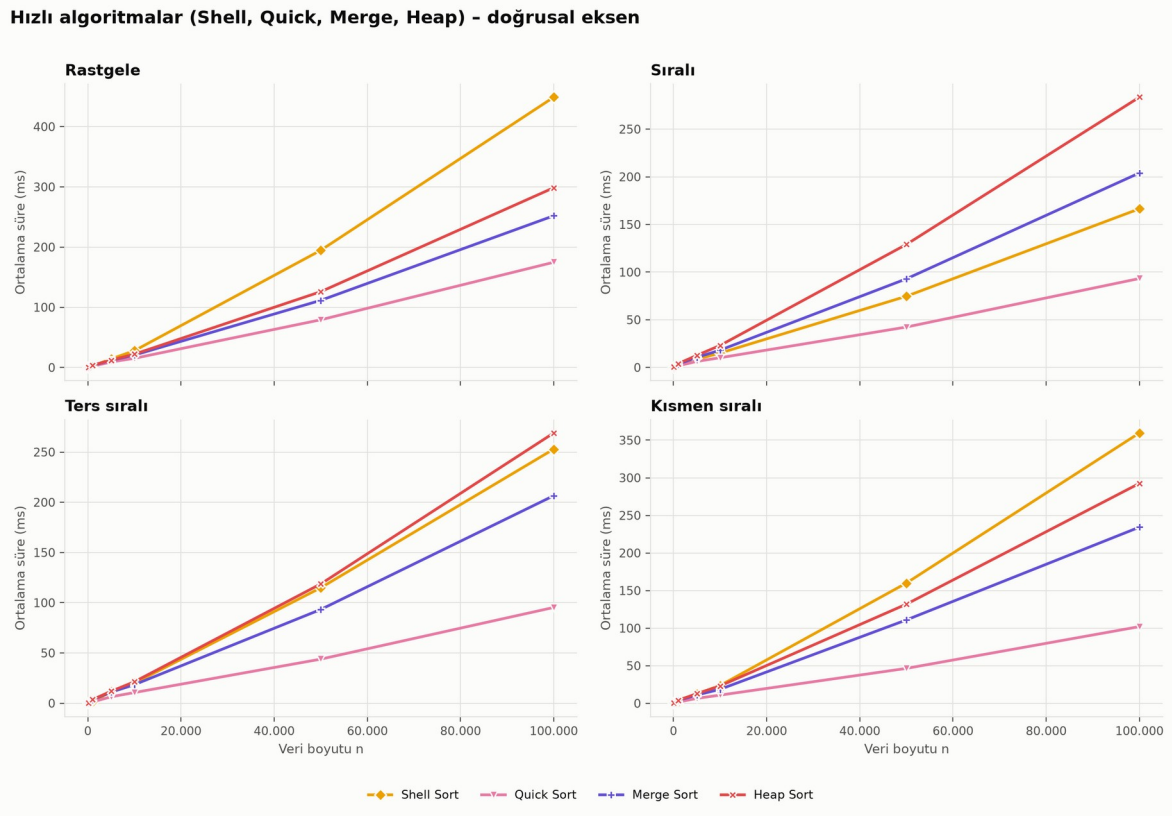
Aşağıdaki şekillerde süre veri boyutuna göre log-log eksende gösterilmiştir. Gri noktalı çizgi $n^2$, kesikli çizgi $n \cdot \log n$ eğilimini temsil eder. Log-log eksende $O(n^2)$ algoritmalar eğimi yaklaşık 2, $O(n \log n)$ algoritmalar yaklaşık 1 olan doğrulara yakındır.



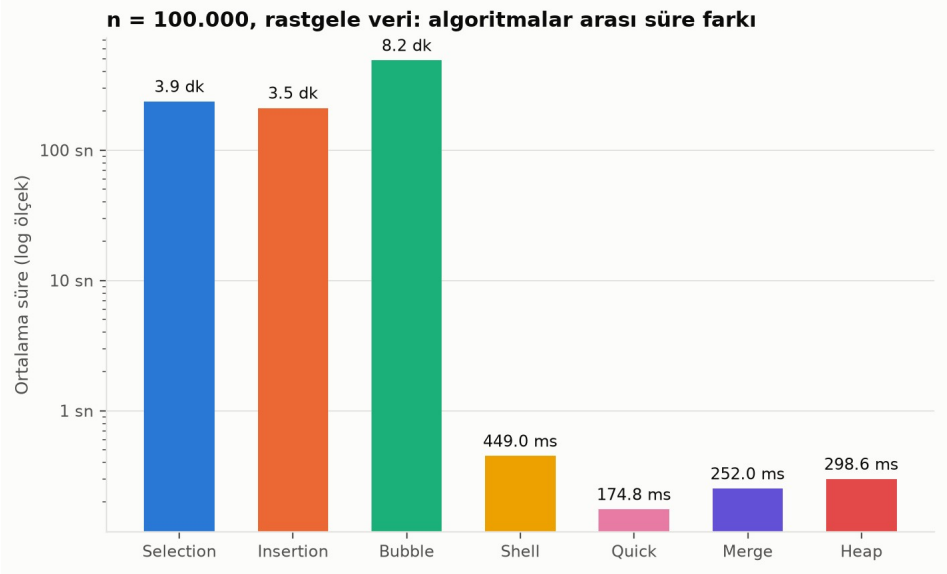

*Şekil 4. Rastgele veride çalışma süresinin veri boyutuna göre değişimi (log-log).*




*Şekil 5. Dört veri dağılımında çalışma süresi (log-log).*



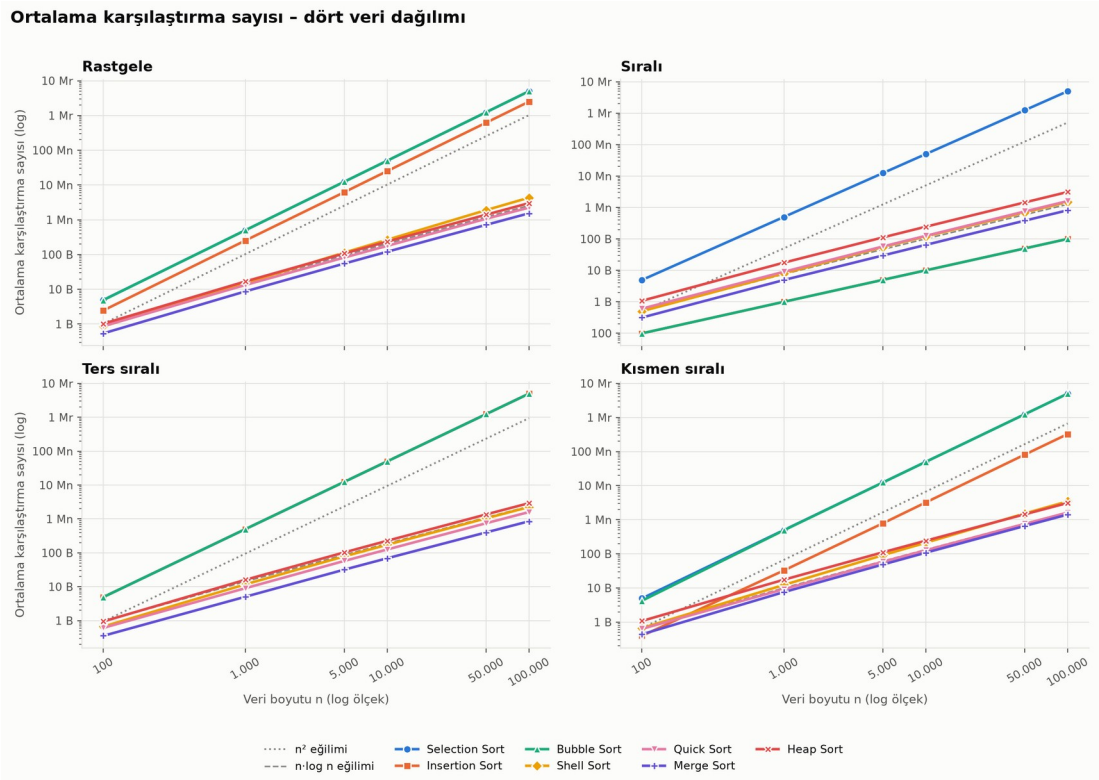

*Şekil 6. Shell, Quick, Merge ve Heap, doğrusal eksen; Shell için n log n garantisi yoktur.*



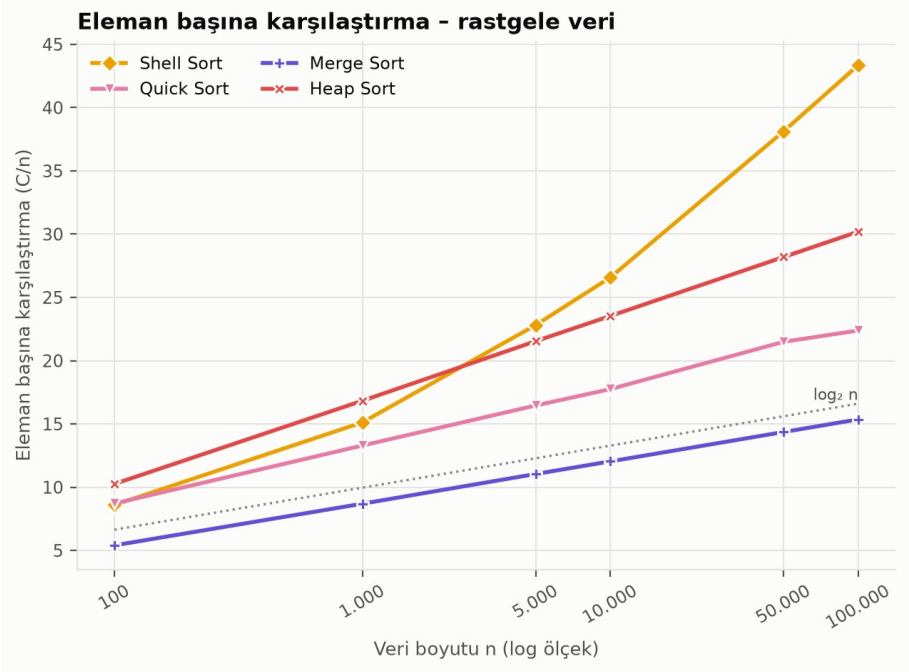

*Şekil 7. n = 100.000 rastgele veride ortalama süre (log eksen).*

Şekil 4–7 süreyi farklı ölçeklerde gösterir. Rastgele veride 100.000 elemanda Quick 174,8 ms, Selection 3,9 dk sürer. Quick'in bölümlemesi ile Selection'ın tüm kalan bölümü taraması arasındaki iş farkı bu ayrımı açıklar. Referans çizgileri yalnızca ölçeklenmiş büyüme eğilimleridir, ölçüm veya performans garantisi değildir.

## 5.4 Karşılaştırma sayısı



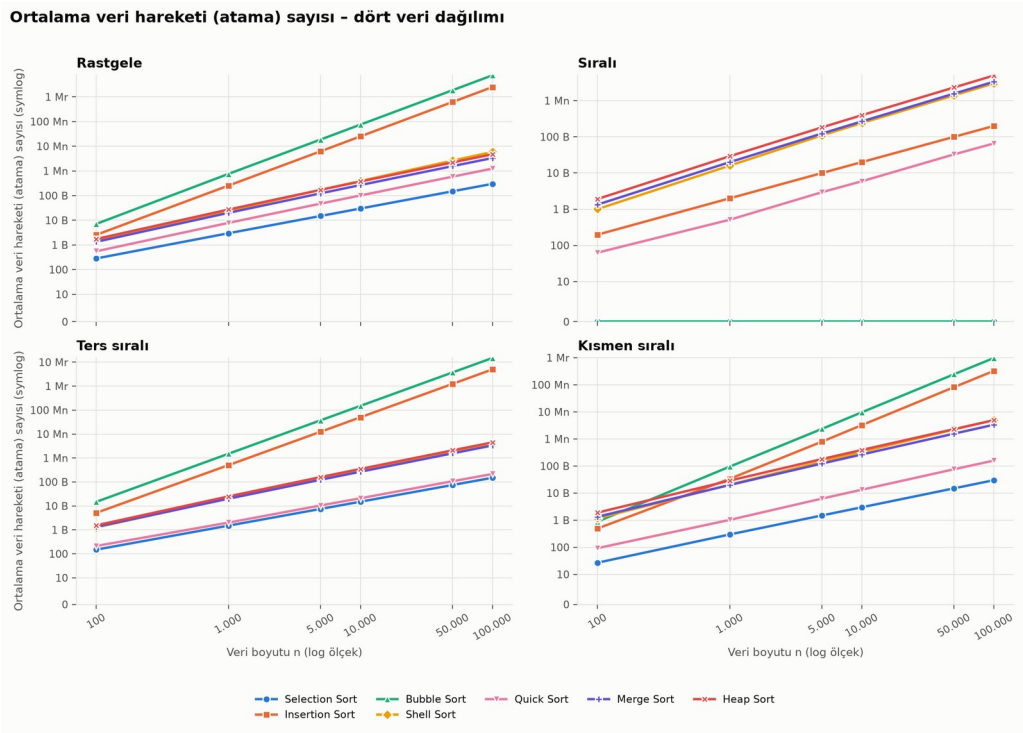

*Şekil 8. Dört veri dağılımında ortalama karşılaştırma sayısı (log-log).*



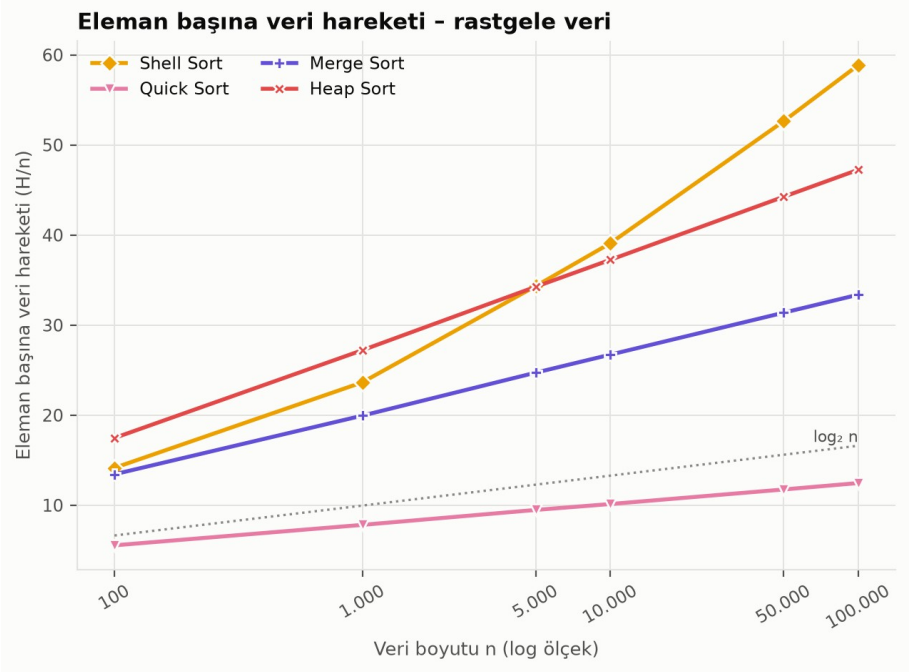

*Şekil 9. Shell, Quick, Merge ve Heap için eleman başına karşılaştırma; $\log_2 n$ referanstır.*

Şekil 8'te rastgele 100.000 eleman için Selection 4.999.950.000, Merge 1.536.353 karşılaştırma yapar. Selection'ın n(n−1)/2 tarama sayısı dağılımdan bağımsızdır; Merge'de birleştirme sırasında yarılardan birinin erken bitmesi karşılaştırma sayısını değiştirir.

## 5.5 Swap / veri hareketi



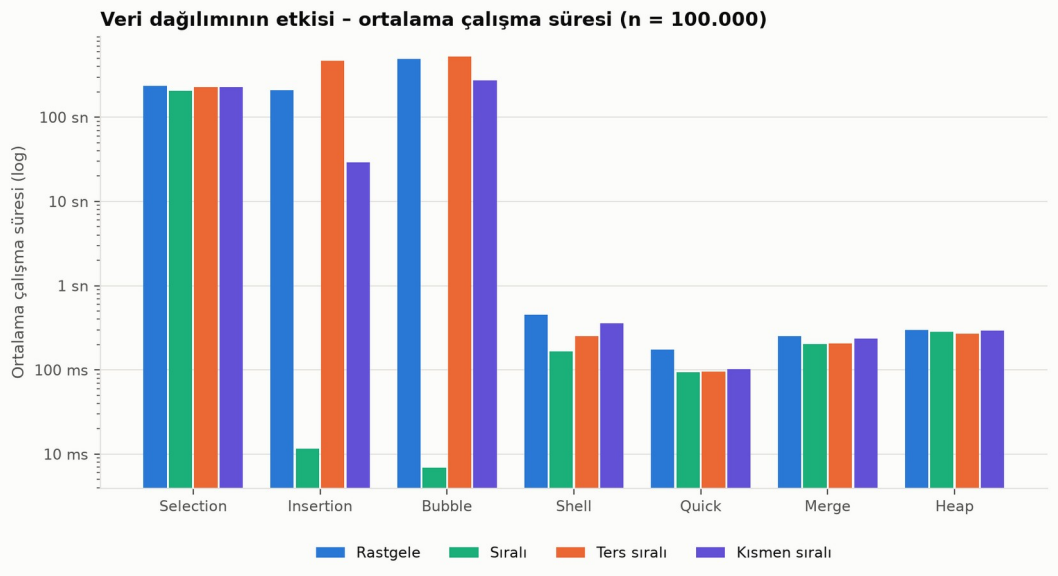

*Şekil 10. Dört veri dağılımında ortalama veri hareketi sayısı (symlog eksen; 0 değerleri gösterilebilir).*



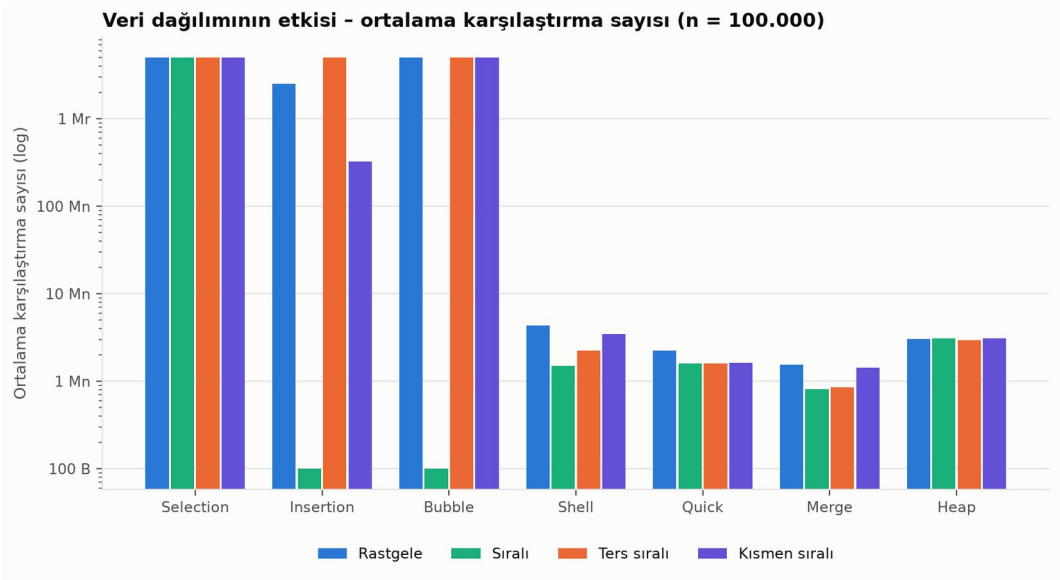

*Şekil 11. Shell, Quick, Merge ve Heap için eleman başına veri hareketi.*

Şekil 10'da 100.000 elemanlı sıralı veride Bubble 0, Insertion 199.998 hareket yapar. Bubble takas yapmadan durur; Insertion ise her elemanı geçiciye alıp tekrar yazar. Symlog ölçeği sıfır hareketi de gösterir. Şekil 11 rastgele girdiye aittir: Quick eleman başına 12,46, Merge 33,38 veri ataması yapar.

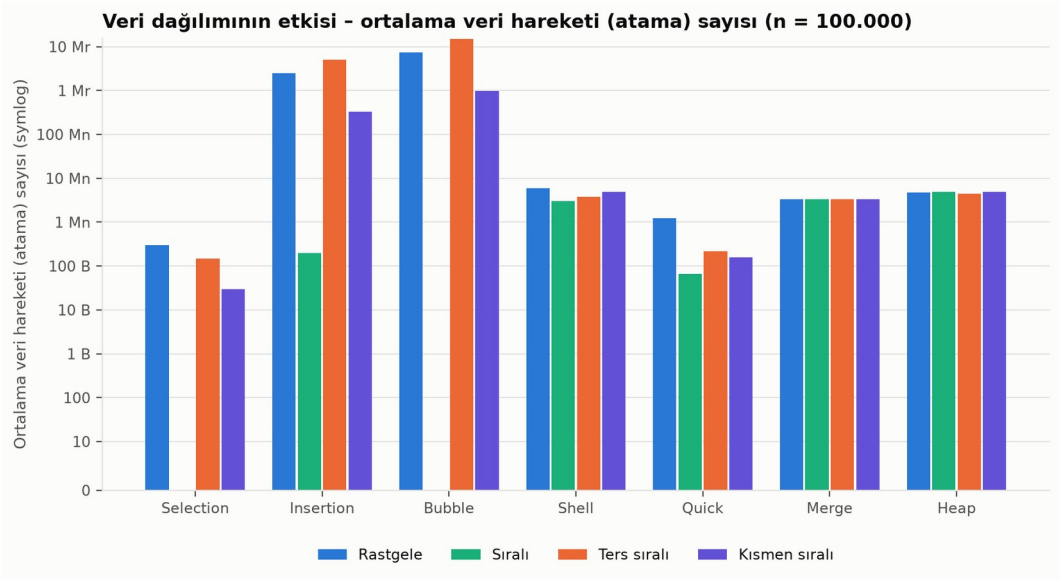
## 5.6 Veri dağılımının performansa etkisi



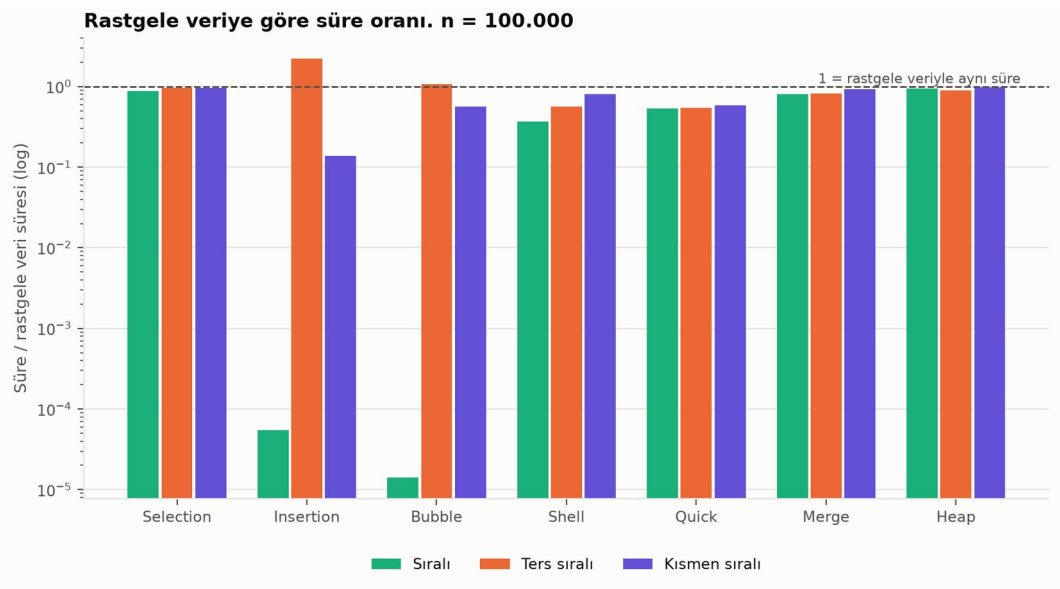

*Şekil 12. Dağılımın çalışma süresine etkisi, n = 100.000 (log eksen).*



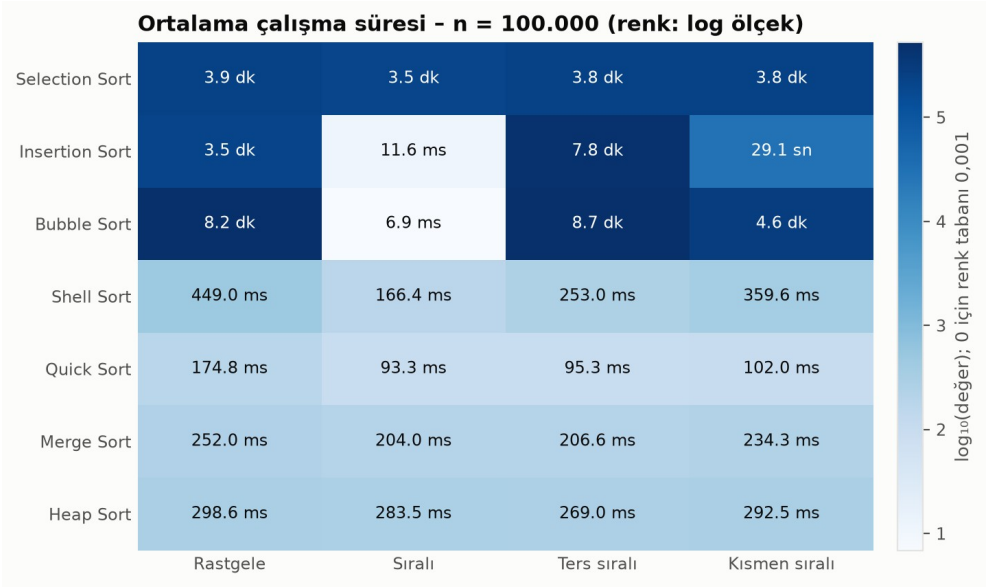

*Şekil 13. Dağılımın karşılaştırma sayısına etkisi, n = 100.000.*



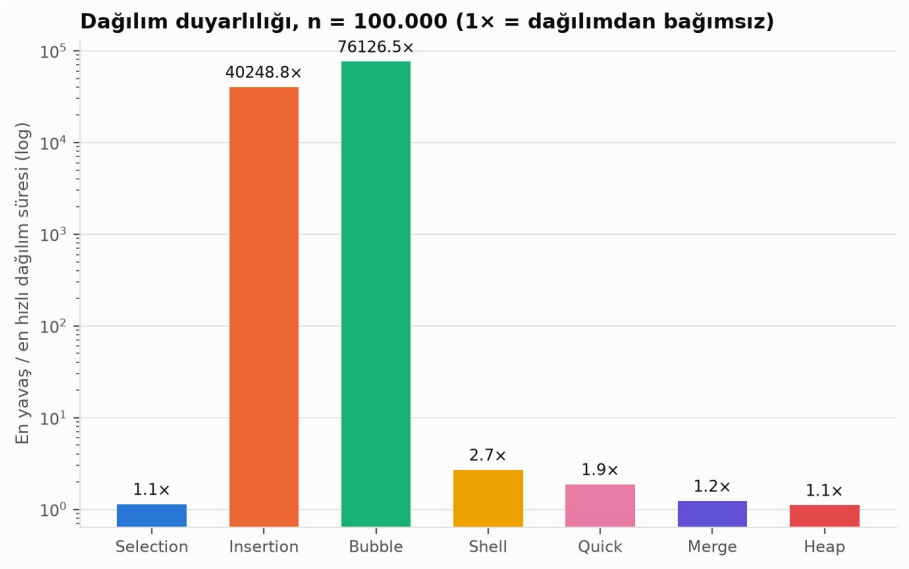

*Şekil 14. Dağılımın veri hareketine etkisi, n = 100.000.*



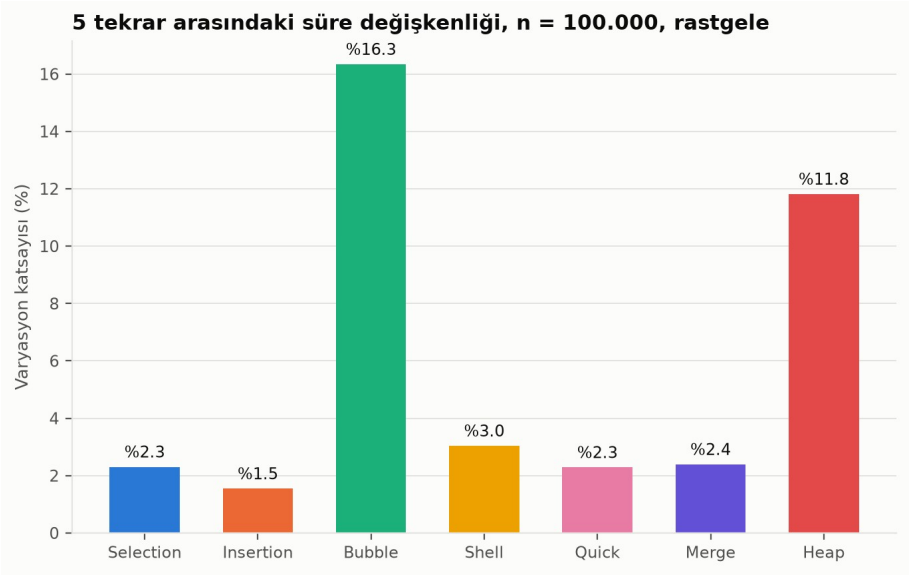

*Şekil 15. Rastgele veriye göre süre oranı, n = 100.000.*




*Şekil 16. Algoritma × dağılım ısı haritası (süre), n = 100.000.*




*Şekil 17. Algoritmaların dağılım duyarlılığı (en yavaş / en hızlı dağılım süresi).*



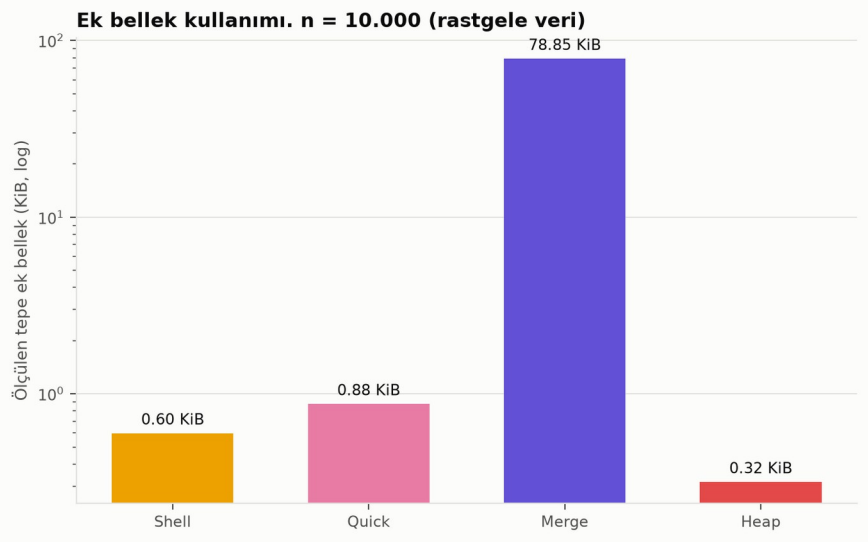

*Şekil 18. Beş tekrar arasındaki süre değişkenliği (varyasyon katsayısı).*

Şekil 12–18 dağılım ve tekrar farklarını ayırır. 100.000 elemanda Insertion sıralı veride 11,6 ms, ters sıralı veride 7,8 dk sürer. Sıralı girdide kaydırma yapılmazken ters sırada her yeni eleman sol bölümün tamamını kaydırır. Beş rastgele tekrar farklı seed kullandığı için son grafikteki değişkenlik hem girdi farkını hem çalışma koşullarını içerir; yalnızca sistem gürültüsü olarak yorumlanamaz. Bubble ve Heap'in değişkenliği diğerlerinden yüksektir. Bu farkın sıcaklıkla ilişkisi kaydedilmediği için böyle bir neden ileri sürülmemiştir.

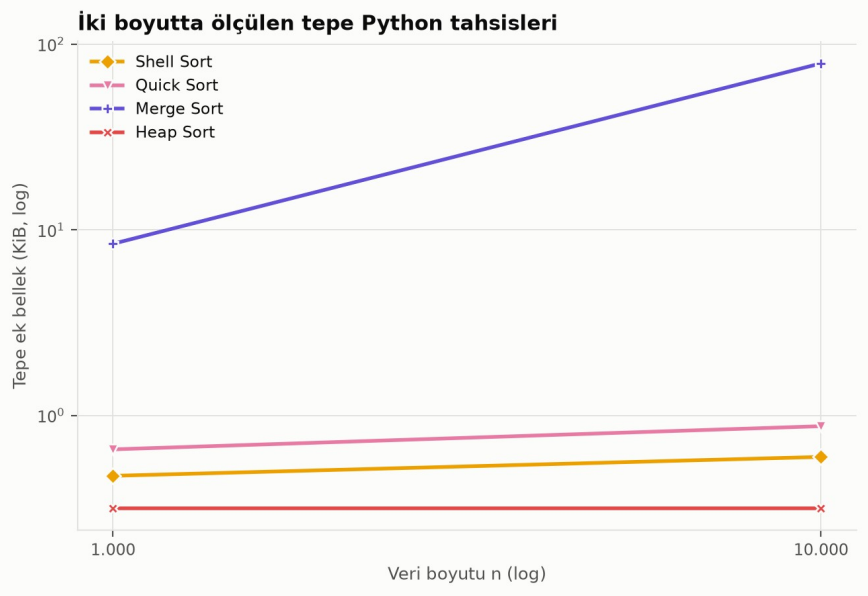
## 5.7 Bellek kullanımı

**Tablo 5. tracemalloc ile ölçülen tepe ek bellek (rastgele veri, bayt)**

| Algoritma | n = 1.000 | n = 10.000 | Teorik ek alan |
|---|---|---|---|
| **Selection** | 348 | ölçülmedi | O(1) |
| **Insertion** | 200 | ölçülmedi | O(1) |
| **Bubble** | 288 | ölçülmedi | O(1) |
| **Shell** | 484 | 612 | O(log n) gap listesi |
| **Quick** | 672 | 896 | O(log n) çağrı yığını üst sınırı |
| **Merge** | 8.612 | 80.740 | O(n) tampon + O(log n) çağrı yığını |
| **Heap** | 324 | 324 | O(1), yinelemeli |




*Şekil 19. Ek bellek kullanımı, n = 10.000 (log eksen).*



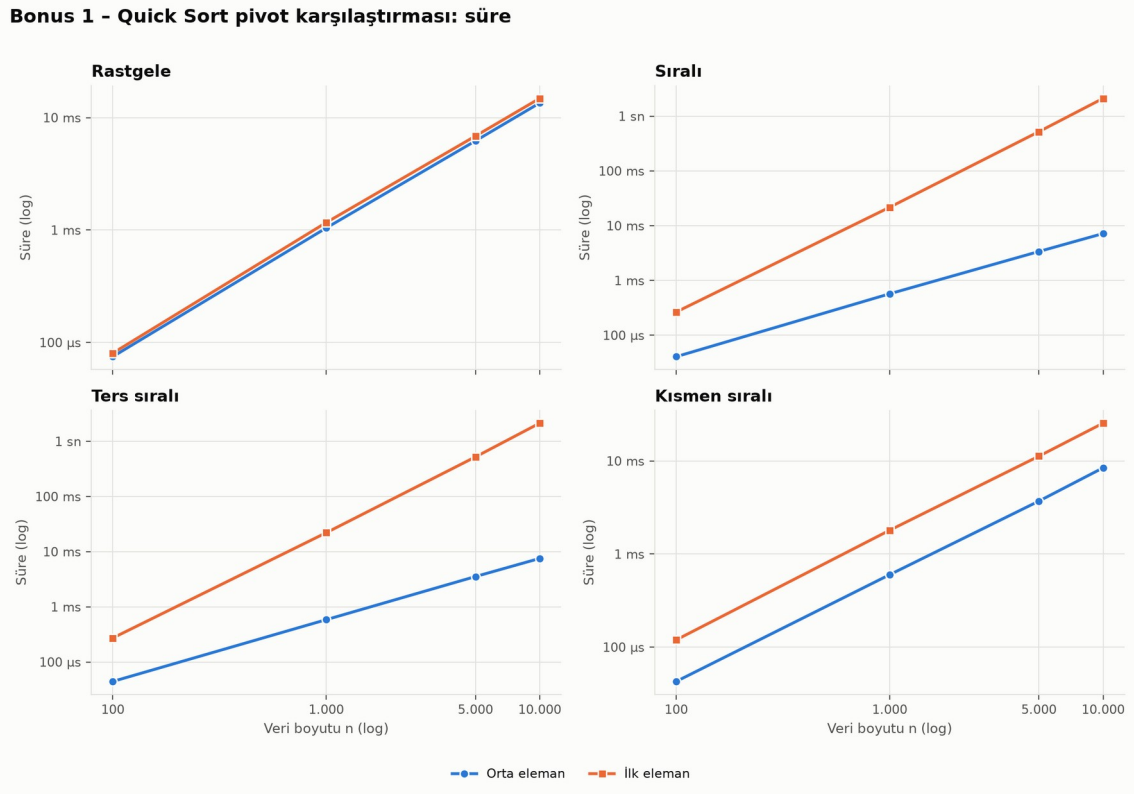

*Şekil 20. Ek belleğin veri boyutuyla büyümesi.*

Şekil 19–20 için rastgele girdide Merge'in tepe tahsisi 8.612 bayttan 80.740 bayta çıkar. Kodda tek n elemanlı tampon bulunması bu artışı açıklar. Quick'in çağrı yığını ve Shell'in gap listesi daha yavaş büyür.

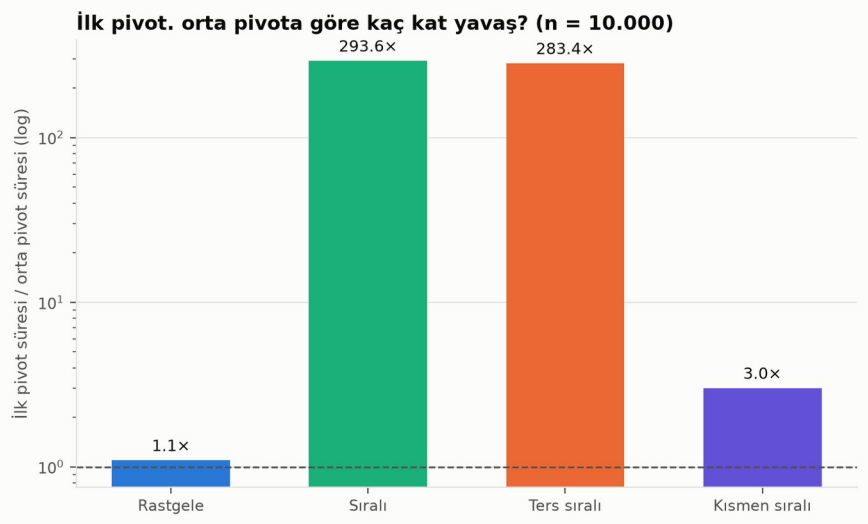
### 5.8 Bonus 1 – Quick Sort pivot karşılaştırması

**Tablo 6. Pivot yöntemleri, n = 10.000**

| Dağılım | Orta (ms) | İlk (ms) | İlk / Orta | Orta karşılaştırma | İlk karşılaştırma |
|---|---|---|---|---|---|
| **Rastgele** | 13,50 | 14,83 | 1,1× | 177.549 | 207.956 |
| **Sıralı** | 7,189 | 2.110,6 | 293,6× | 125.439 | 50.014.998 |
| **Ters sıralı** | 7,535 | 2.135,2 | 283,4× | 125.452 | 50.019.996 |
| **Kısmen sıralı** | 8,445 | 25,48 | 3,0× | 127.163 | 494.175 |




*Şekil 21. Pivot yöntemlerinin süreye etkisi.*

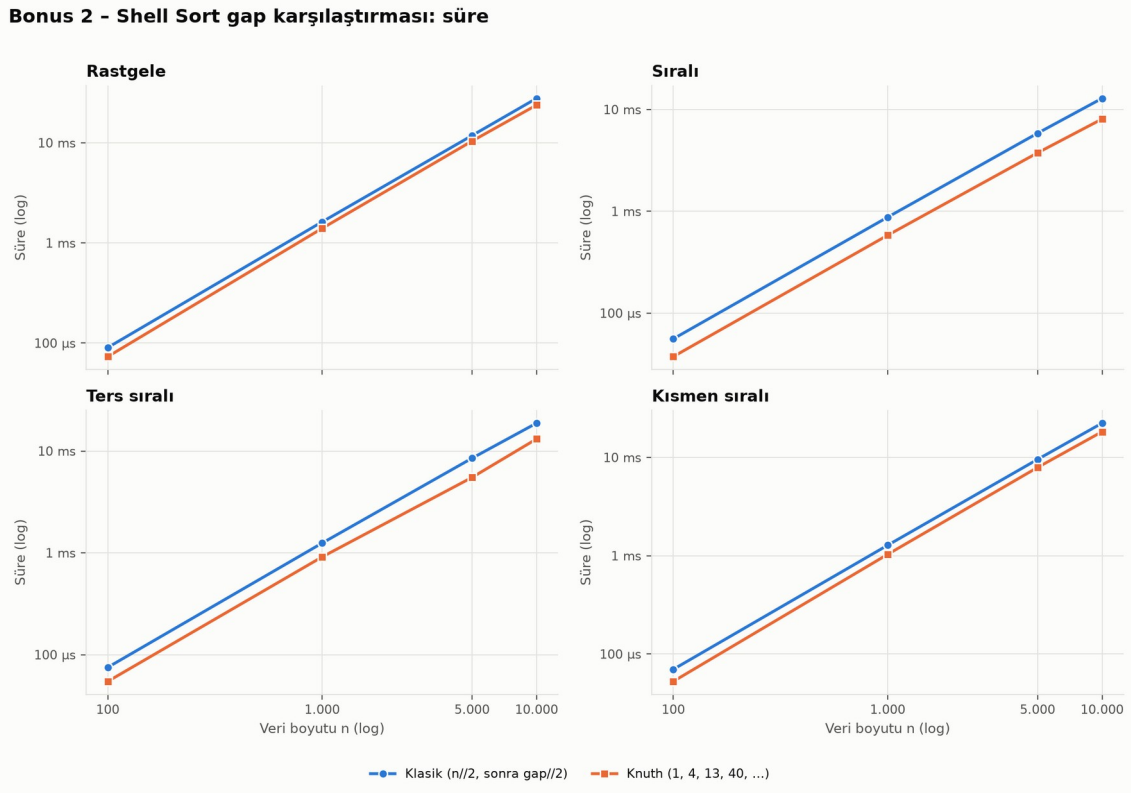



*Şekil 22. İlk pivot süresinin orta pivot süresine oranı, n = 10.000.*

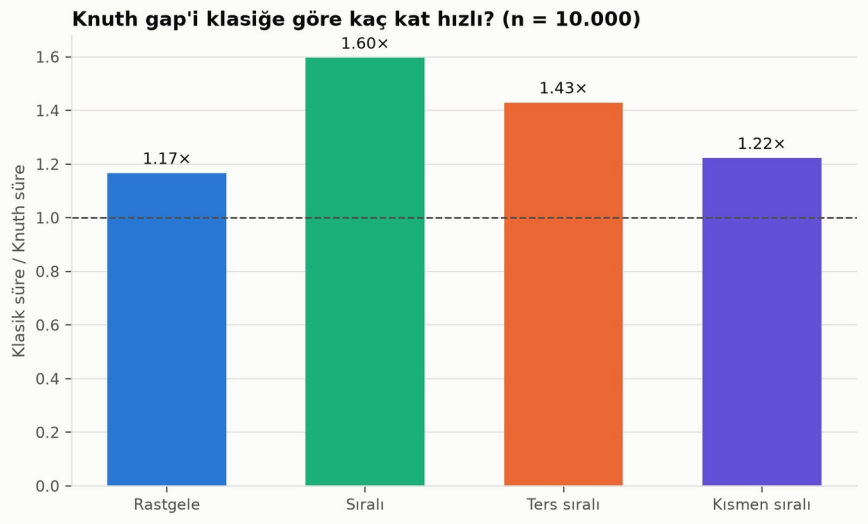
## 5.9 Bonus 2 – Shell Sort gap karşılaştırması

**Tablo 7. Gap dizileri, n = 10.000**

| Dağılım | Klasik (ms) | Knuth (ms) | Klasik / Knuth | Klasik karşılaştırma | Knuth karşılaştırma |
|---|---|---|---|---|---|
| **Rastgele** | 27,79 | 23,82 | 1,17× | 265.793 | 239.323 |
| **Sıralı** | 12,87 | 8,064 | 1,60× | 120.005 | 75.243 |
| **Ters sıralı** | 18,77 | 13,13 | 1,43× | 172.578 | 120.190 |
| **Kısmen sıralı** | 22,30 | 18,24 | 1,22× | 211.738 | 180.081 |




*Şekil 23. Gap dizilerinin süreye etkisi.*

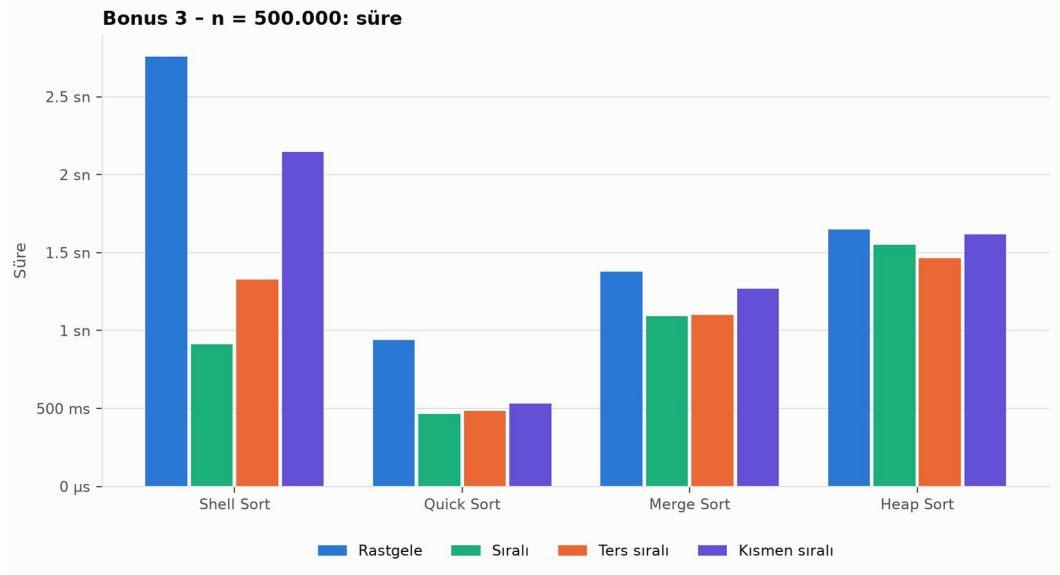



*Şekil 24. Klasik gap süresinin Knuth gap süresine oranı, n = 10.000.*

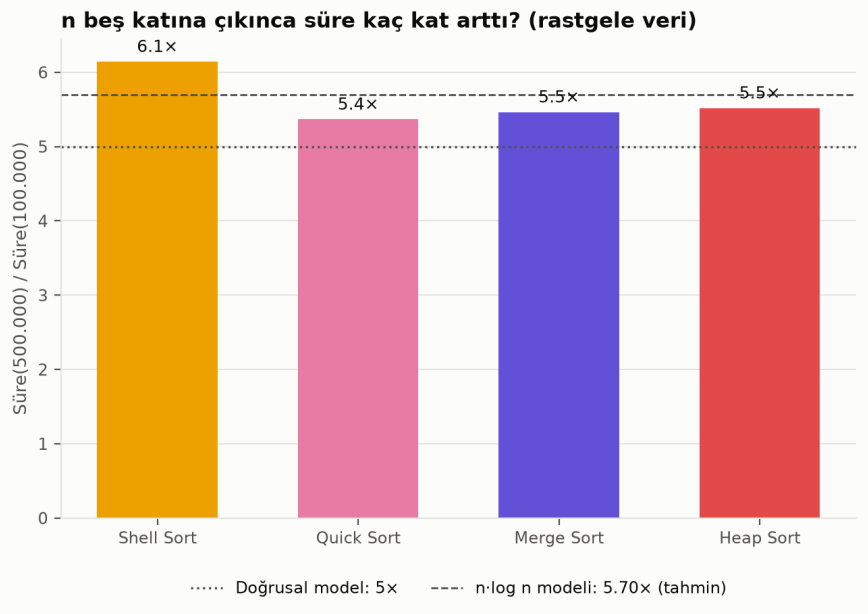
## 5.10 Bonus 3 – n = 500.000

**Tablo 8. n = 500.000 için ortalama süre**

| Dağılım | Shell | Quick | Merge | Heap |
|---|---|---|---|---|
| **Rastgele** | 2,8 sn | 938,0 ms | 1,4 sn | 1,6 sn |
| **Sıralı** | 912,0 ms | 466,4 ms | 1,1 sn | 1,6 sn |
| **Ters sıralı** | 1,3 sn | 483,9 ms | 1,1 sn | 1,5 sn |
| **Kısmen sıralı** | 2,1 sn | 530,2 ms | 1,3 sn | 1,6 sn |




*Şekil 25. n = 500.000'de süre karşılaştırması.*

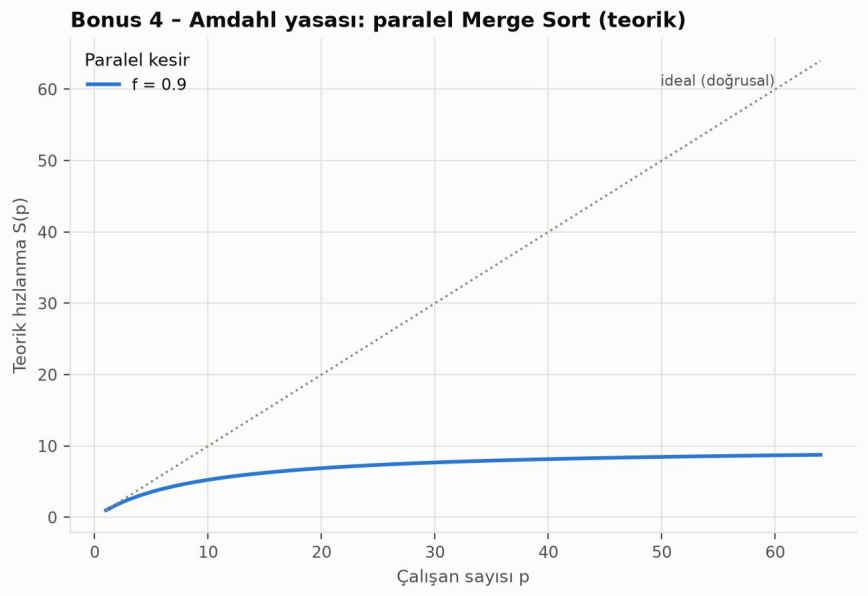



*Şekil 26. n 100.000'den 500.000'e çıkınca sürenin büyüme çarpanı.*

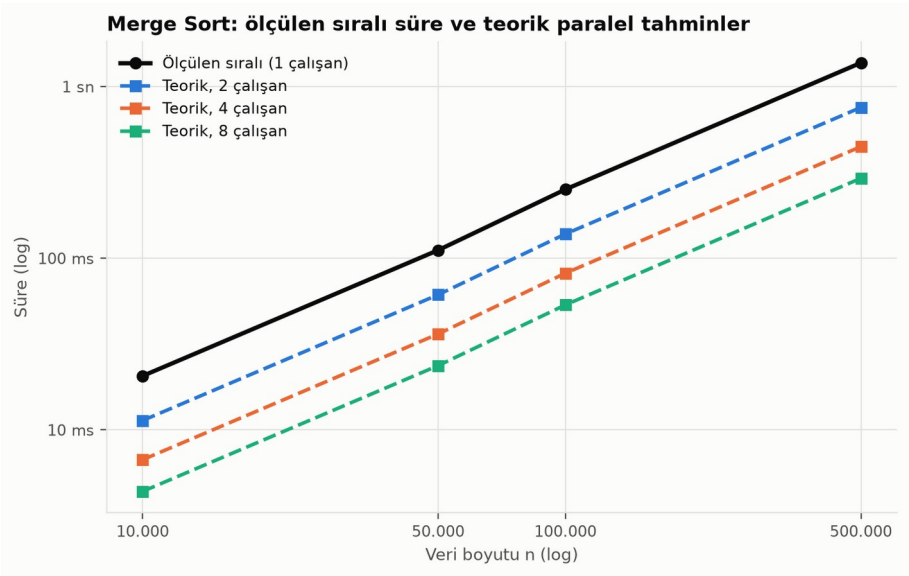
Şekil 26 rastgele girdide beş kat boyut artışının gerçek süre oranlarını gösterir: Shell 6,14 kat; Quick 5,37 kat; Merge 5,46 kat; Heap 5,52 kat. n·log n modeli bu aralıkta yaklaşık 5,70 kat artış verir. Model, aynı sabit maliyet varsayımı altında bir referanstır; oturum farklarını içermez. Shell'in oranı yalnızca gözlemdir ve n log n üst sınırı anlamına gelmez.

### 5.11 Bonus 4 – Paralel Merge Sort (teorik)

Paralel Merge Sort yalnızca teorik olarak incelenmiştir; paralel kod yazılmamış, paralel ölçüm yapılmamıştır. Aşağıdaki iş ve kritik yol değerlendirmesi böl ve yönet modeline dayanır (Cormen ve diğerleri, 2022). Sıralı Merge Sort'ta sol ve sağ yarının sıralanması bağımsızdır ve eşzamanlı yürütülebilir; üst düzey birleştirme ise iki yarının tamamlanmasını bekler. Toplam iş $T_1(n) = \Theta(n \log n)$; alt problemler paralel ama birleştirme sıralı olduğunda kritik yol $T\infty(n) = T\infty(n/2) + \Theta(n) = \Theta(n)$ olur. Teorik paralellik $T_1/T\infty = \Theta(\log n)$ ile sınırlıdır: çok sayıda çekirdek kullanılsa bile sıralı birleştirme darboğaz oluşturur. Amdahl yasası (Amdahl, 1967), paralel kesri f olan bir iş için $S(p) = 1 / ((1 - f) + f/p)$ hızlanma verir; f = 0,9 için 2, 4 ve 8 çalışanla teorik hızlanma sırasıyla yaklaşık 1,82; 3,08 ve 4,71'dir. Bu değerler gerçek ölçüm değil, idealize bir model sonucudur. Ayrıca bu çalışmanın Python 3.13 ortamında GIL etkindir; saf Python thread'lerinin CPU ağırlıklı sıralamada paralel bayt kodu yürütmesi beklenmez, çok çekirdekli kazanç için ayrı süreçler kullanılabilir (Python Software Foundation, threading belgeleri). Süreç başlatma ve veri aktarım maliyetleri bu çalışmada ölçülmemiştir.




*Şekil 27. Amdahl yasasına göre teorik hızlanma (gerçek ölçüm değildir).*




*Şekil 28. Ölçülen sıralı süre ve Amdahl modelinden hesaplanan teorik paralel süreler.*

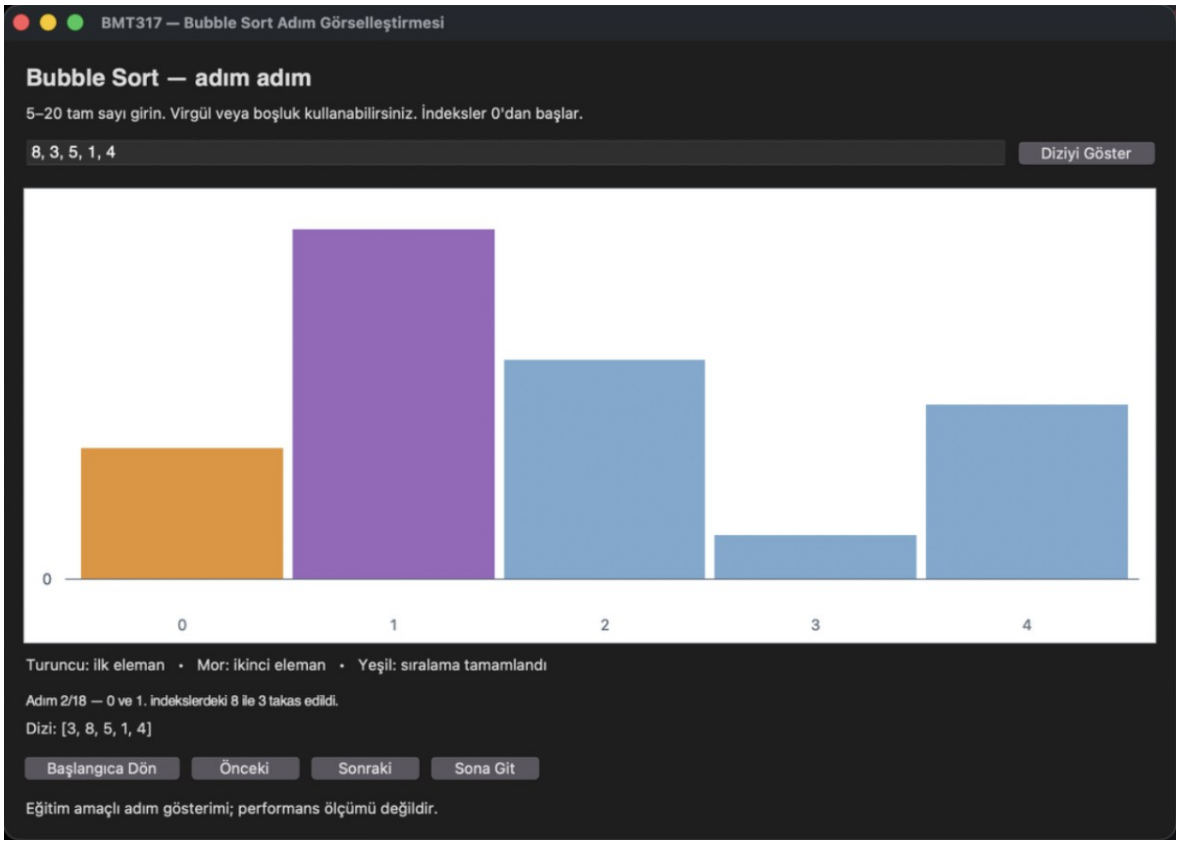
### 5.12 Bonus 5 – Bubble Sort adım görselleştirmesi

gorsellestirme/bubble_gorsel.py Tkinter ile yazılmış bağımsız bir eğitim uygulamasıdır. 5–20 tam sayıdan oluşan bir dizi girilir; **Sonraki**, **Önceki**, **Başlangıca Dön** ve **Sona Git** düğmeleriyle Bubble Sort'un karşılaştırma ve takas adımları sütunlar üzerinde izlenir. Uygulama performans ölçümü yapmaz. Adım mantığı birim testleriyle doğrulanmıştır.

Uygulama PyCharm üzerinden çalıştırıldı ve adım geçişleri tarafımdan kontrol edildi. Şekil 29 başlangıç dizisini, Şekil 30 ilk takası, Şekil 31 tamamlanmış sıralamayı gösterir. Görseller kendi ekran görüntülerimden yalnızca kırpılarak alınmıştır; görüntü içeriği değiştirilmemiştir.




*Şekil 29. Bubble Sort başlangıcı: [8, 3, 5, 1, 4] dizisi, adım 0/18.*




*Şekil 30. İlk takas: 0 ve 1. indekslerdeki 8 ile 3 yer değiştirir; dizi [3, 8, 5, 1, 4] olur (adım 2/18).*




*Şekil 31. Tamamlanmış sıralama: [1, 3, 4, 5, 8]. Takassız son tur erken durmayı sağlar (adım 18/18).*

### 5.13 Tüm zorunlu deney sonuçları (ortalama, 5 tekrar)

Aşağıdaki tablo tüm zorunlu kombinasyonları içerir; ham ölçümler (ham_<n>.csv) ve özetler (ozet_<n>.csv) ayrıca CSV olarak teslim edilmiştir.

**Tablo 9. Tüm kombinasyonlar için ortalama süre, karşılaştırma ve hareket sayısı**

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Selection | Rastgele | 100 | 0,190 | 4.950 | 284 |
| Insertion | Rastgele | 100 | 0,250 | 2.471 | 2.575 |
| Bubble | Rastgele | 100 | 1,150 | 4.895 | 7.132 |
| Shell | Rastgele | 100 | 0,428 | 860 | 1.415 |
| Quick | Rastgele | 100 | 0,388 | 872 | 555 |
| Merge | Rastgele | 100 | 0,582 | 541 | 1.344 |
| Heap | Rastgele | 100 | 0,256 | 1.026 | 1.747 |
| Selection | Sıralı | 100 | 1,030 | 4.950 | 0 |
| Insertion | Sıralı | 100 | 0,080 | 99 | 198 |
| Bubble | Sıralı | 100 | 0,032 | 99 | 0 |
| Shell | Sıralı | 100 | 0,356 | 503 | 1.006 |
| Quick | Sıralı | 100 | 0,119 | 606 | 63 |
| Merge | Sıralı | 100 | 0,503 | 316 | 1.344 |
| Heap | Sıralı | 100 | 0,553 | 1.081 | 1.920 |
| Selection | Ters sıralı | 100 | 0,771 | 4.950 | 150 |
| Insertion | Ters sıralı | 100 | 1,284 | 4.950 | 5.148 |
| Bubble | Ters sıralı | 100 | 1,427 | 4.950 | 14.850 |
| Shell | Ters sıralı | 100 | 0,521 | 668 | 1.266 |
| Quick | Ters sıralı | 100 | 0,108 | 610 | 213 |
| Merge | Ters sıralı | 100 | 0,312 | 356 | 1.344 |
| Heap | Ters sıralı | 100 | 0,475 | 944 | 1.548 |
| Selection | Kısmen sıralı | 100 | 1,273 | 4.950 | 27 |
| Insertion | Kısmen sıralı | 100 | 0,088 | 396 | 495 |
| Bubble | Kısmen sıralı | 100 | 0,534 | 4.223 | 891 |
| Shell | Kısmen sıralı | 100 | 0,467 | 667 | 1.176 |
| Quick | Kısmen sıralı | 100 | 0,292 | 633 | 93 |

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Merge | Kısmen sıralı | 100 | 0,347 | 438 | 1.344 |
| Heap | Kısmen sıralı | 100 | 0,353 | 1.079 | 1.907 |
| Selection | Rastgele | 1.000 | 21,27 | 499.500 | 2.977 |
| Insertion | Rastgele | 1.000 | 20,88 | 252.946 | 253.953 |
| Bubble | Rastgele | 1.000 | 34,71 | 498.789 | 755.865 |
| Shell | Rastgele | 1.000 | 2,458 | 15.125 | 23.651 |
| Quick | Rastgele | 1.000 | 1,964 | 13.295 | 7.823 |
| Merge | Rastgele | 1.000 | 3,225 | 8.698 | 19.952 |
| Heap | Rastgele | 1.000 | 3,799 | 16.834 | 27.217 |
| Selection | Sıralı | 1.000 | 20,33 | 499.500 | 0 |
| Insertion | Sıralı | 1.000 | 0,160 | 999 | 1.998 |
| Bubble | Sıralı | 1.000 | 0,100 | 999 | 0 |
| Shell | Sıralı | 1.000 | 2,518 | 8.006 | 16.012 |
| Quick | Sıralı | 1.000 | 1,662 | 9.009 | 511 |
| Merge | Sıralı | 1.000 | 3,441 | 4.932 | 19.952 |
| Heap | Sıralı | 1.000 | 3,936 | 17.583 | 29.124 |
| Selection | Ters sıralı | 1.000 | 21,58 | 499.500 | 1.500 |
| Insertion | Ters sıralı | 1.000 | 41,70 | 499.500 | 501.498 |
| Bubble | Ters sıralı | 1.000 | 46,20 | 499.500 | 1.498.500 |
| Shell | Ters sıralı | 1.000 | 1,414 | 11.716 | 20.712 |
| Quick | Ters sıralı | 1.000 | 1,231 | 9.016 | 2.011 |
| Merge | Ters sıralı | 1.000 | 3,429 | 5.044 | 19.952 |
| Heap | Ters sıralı | 1.000 | 3,612 | 15.965 | 24.948 |
| Selection | Kısmen sıralı | 1.000 | 21,35 | 499.500 | 297 |
| Insertion | Kısmen sıralı | 1.000 | 3,748 | 32.654 | 33.653 |
| Bubble | Kısmen sıralı | 1.000 | 23,77 | 490.548 | 94.965 |
| Shell | Kısmen sıralı | 1.000 | 2,159 | 12.273 | 20.351 |
| Quick | Kısmen sıralı | 1.000 | 1,587 | 9.206 | 1.025 |
| Merge | Kısmen sıralı | 1.000 | 3,743 | 7.567 | 19.952 |
| Heap | Kısmen sıralı | 1.000 | 4,039 | 17.485 | 28.958 |
| Selection | Rastgele | 5.000 | 485,12 | 12.497.500 | 14.969 |

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Insertion | Rastgele | 5.000 | 512,91 | 6.255.973 | 6.260.980 |
| Bubble | Rastgele | 5.000 | 903,38 | 12.494.591 | 18.752.947 |
| Shell | Rastgele | 5.000 | 14,21 | 114.066 | 171.650 |
| Quick | Rastgele | 5.000 | 9,156 | 82.307 | 47.410 |
| Merge | Rastgele | 5.000 | 11,79 | 55.234 | 123.616 |
| Heap | Rastgele | 5.000 | 12,18 | 107.709 | 171.293 |
| Selection | Sıralı | 5.000 | 461,34 | 12.497.500 | 0 |
| Insertion | Sıralı | 5.000 | 1,766 | 4.999 | 9.998 |
| Bubble | Sıralı | 5.000 | 0,706 | 4.999 | 0 |
| Shell | Sıralı | 5.000 | 8,583 | 55.005 | 110.010 |
| Quick | Sıralı | 5.000 | 6,313 | 57.726 | 2.952 |
| Merge | Sıralı | 5.000 | 10,49 | 29.804 | 123.616 |
| Heap | Sıralı | 5.000 | 12,64 | 112.126 | 182.796 |
| Selection | Ters sıralı | 5.000 | 497,05 | 12.497.500 | 7.500 |
| Insertion | Ters sıralı | 5.000 | 994,06 | 12.497.500 | 12.507.498 |
| Bubble | Ters sıralı | 5.000 | 1.150,2 | 12.497.500 | 37.492.500 |
| Shell | Ters sıralı | 5.000 | 11,44 | 78.798 | 138.790 |
| Quick | Ters sıralı | 5.000 | 6,384 | 57.738 | 10.453 |
| Merge | Ters sıralı | 5.000 | 11,00 | 32.004 | 123.616 |
| Heap | Ters sıralı | 5.000 | 11,87 | 103.227 | 160.308 |
| Selection | Kısmen sıralı | 5.000 | 480,96 | 12.497.500 | 1.497 |
| Insertion | Kısmen sıralı | 5.000 | 62,95 | 791.555 | 796.555 |
| Bubble | Kısmen sıralı | 5.000 | 573,10 | 12.411.904 | 2.359.670 |
| Shell | Kısmen sıralı | 5.000 | 12,44 | 91.803 | 147.151 |
| Quick | Kısmen sıralı | 5.000 | 6,541 | 58.829 | 6.283 |
| Merge | Kısmen sıralı | 5.000 | 10,99 | 48.917 | 123.616 |
| Heap | Kısmen sıralı | 5.000 | 12,78 | 111.664 | 181.142 |
| Selection | Rastgele | 10.000 | 1.926,1 | 49.995.000 | 29.965 |
| Insertion | Rastgele | 10.000 | 2.025,8 | 25.137.257 | 25.147.264 |
| Bubble | Rastgele | 10.000 | 3.629,6 | 49.990.762 | 75.381.798 |
| Shell | Rastgele | 10.000 | 27,86 | 265.793 | 390.899 |

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Quick | Rastgele | 10.000 | 15,23 | 177.549 | 101.361 |
| Merge | Rastgele | 10.000 | 20,64 | 120.414 | 267.232 |
| Heap | Rastgele | 10.000 | 22,13 | 235.358 | 372.504 |
| Selection | Sıralı | 10.000 | 1.882,2 | 49.995.000 | 0 |
| Insertion | Sıralı | 10.000 | 2,578 | 9.999 | 19.998 |
| Bubble | Sıralı | 10.000 | 1,110 | 9.999 | 0 |
| Shell | Sıralı | 10.000 | 15,06 | 120.005 | 240.010 |
| Quick | Sıralı | 10.000 | 10,18 | 125.439 | 5.904 |
| Merge | Sıralı | 10.000 | 18,12 | 64.608 | 267.232 |
| Heap | Sıralı | 10.000 | 23,03 | 244.460 | 395.868 |
| Selection | Ters sıralı | 10.000 | 1.989,8 | 49.995.000 | 15.000 |
| Insertion | Ters sıralı | 10.000 | 4.008,1 | 49.995.000 | 50.014.998 |
| Bubble | Ters sıralı | 10.000 | 4.590,0 | 49.995.000 | 149.985.000 |
| Shell | Ters sıralı | 10.000 | 19,91 | 172.578 | 302.570 |
| Quick | Ters sıralı | 10.000 | 10,60 | 125.452 | 20.905 |
| Merge | Ters sıralı | 10.000 | 18,42 | 69.008 | 267.232 |
| Heap | Ters sıralı | 10.000 | 21,33 | 226.682 | 350.088 |
| Selection | Kısmen sıralı | 10.000 | 1.912,6 | 49.995.000 | 2.997 |
| Insertion | Kısmen sıralı | 10.000 | 256,05 | 3.221.009 | 3.231.008 |
| Bubble | Kısmen sıralı | 10.000 | 2.367,9 | 49.880.040 | 9.633.029 |
| Shell | Kısmen sıralı | 10.000 | 23,65 | 211.738 | 332.445 |
| Quick | Kısmen sıralı | 10.000 | 10,89 | 127.163 | 13.332 |
| Merge | Kısmen sıralı | 10.000 | 18,60 | 108.166 | 267.232 |
| Heap | Kısmen sıralı | 10.000 | 23,11 | 243.422 | 392.311 |
| Selection | Rastgele | 50.000 | 51.564,1 | 1.249.975.000 | 149.963 |
| Insertion | Rastgele | 50.000 | 52.551,5 | 625.987.157 | 626.037.168 |
| Bubble | Rastgele | 50.000 | 98.406,1 | 1.249.957.356 | 1.877.811.511 |
| Shell | Rastgele | 50.000 | 194,67 | 1.906.309 | 2.631.789 |
| Quick | Rastgele | 50.000 | 79,24 | 1.074.603 | 587.629 |
| Merge | Rastgele | 50.000 | 111,53 | 718.211 | 1.568.928 |
| Heap | Rastgele | 50.000 | 125,93 | 1.409.763 | 2.212.417 |

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Selection | Sıralı | 50.000 | 52.394,0 | 1.249.975.000 | 0 |
| Insertion | Sıralı | 50.000 | 7,160 | 49.999 | 99.998 |
| Bubble | Sıralı | 50.000 | 3,167 | 49.999 | 0 |
| Shell | Sıralı | 50.000 | 74,69 | 700.006 | 1.400.012 |
| Quick | Sıralı | 50.000 | 42,38 | 750.015 | 32.767 |
| Merge | Sıralı | 50.000 | 92,97 | 382.512 | 1.568.928 |
| Heap | Sıralı | 50.000 | 129,07 | 1.455.438 | 2.319.912 |
| Selection | Ters sıralı | 50.000 | 60.086,7 | 1.249.975.000 | 75.000 |
| Insertion | Ters sıralı | 50.000 | 115.744,4 | 1.249.975.000 | 1.250.074.998 |
| Bubble | Ters sıralı | 50.000 | 126.812,7 | 1.249.975.000 | 3.749.925.000 |
| Shell | Ters sıralı | 50.000 | 114,91 | 1.047.305 | 1.797.292 |
| Quick | Ters sıralı | 50.000 | 43,87 | 750.028 | 107.767 |
| Merge | Ters sıralı | 50.000 | 93,32 | 401.952 | 1.568.928 |
| Heap | Ters sıralı | 50.000 | 118,74 | 1.366.047 | 2.096.676 |
| Selection | Kısmen sıralı | 50.000 | 51.438,6 | 1.249.975.000 | 14.997 |
| Insertion | Kısmen sıralı | 50.000 | 7.030,3 | 81.849.712 | 81.899.712 |
| Bubble | Kısmen sıralı | 50.000 | 65.754,3 | 1.249.675.762 | 245.399.141 |
| Shell | Kısmen sıralı | 50.000 | 159,95 | 1.531.970 | 2.235.613 |
| Quick | Kısmen sıralı | 50.000 | 46,47 | 759.796 | 76.011 |
| Merge | Kısmen sıralı | 50.000 | 110,91 | 659.692 | 1.568.928 |
| Heap | Kısmen sıralı | 50.000 | 131,87 | 1.451.033 | 2.312.961 |
| Selection | Rastgele | 100.000 | 236.668,8 | 4.999.950.000 | 299.966 |
| Insertion | Rastgele | 100.000 | 210.804,3 | 2.505.370.820 | 2.505.470.830 |
| Bubble | Rastgele | 100.000 | 489.305,1 | 4.999.914.055 | 7.515.812.496 |
| Shell | Rastgele | 100.000 | 448,97 | 4.335.792 | 5.886.484 |
| Quick | Rastgele | 100.000 | 174,78 | 2.238.729 | 1.246.496 |
| Merge | Rastgele | 100.000 | 252,01 | 1.536.353 | 3.337.856 |
| Heap | Rastgele | 100.000 | 298,59 | 3.019.566 | 4.724.852 |
| Selection | Sıralı | 100.000 | 207.822,5 | 4.999.950.000 | 0 |
| Insertion | Sıralı | 100.000 | 11,61 | 99.999 | 199.998 |
| Bubble | Sıralı | 100.000 | 6,896 | 99.999 | 0 |

| Algoritma | Veri tipi | n | Ort. süre (ms) | Karşılaştırma | Swap/ Hareket |
|---|---|---|---|---|---|
| Shell | Sıralı | 100.000 | 166,44 | 1.500.006 | 3.000.012 |
| Quick | Sıralı | 100.000 | 93,31 | 1.600.016 | 65.535 |
| Merge | Sıralı | 100.000 | 204,04 | 815.024 | 3.337.856 |
| Heap | Sıralı | 100.000 | 283,53 | 3.112.517 | 4.952.562 |
| Selection | Ters sıralı | 100.000 | 228.004,4 | 4.999.950.000 | 150.000 |
| Insertion | Ters sıralı | 100.000 | 467.263,2 | 4.999.950.000 | 5.000.149.998 |
| Bubble | Ters sıralı | 100.000 | 524.991,0 | 4.999.950.000 | 14.999.850.000 |
| Shell | Ters sıralı | 100.000 | 252,95 | 2.244.585 | 3.844.572 |
| Quick | Ters sıralı | 100.000 | 95,33 | 1.600.030 | 215.535 |
| Merge | Ters sıralı | 100.000 | 206,60 | 853.904 | 3.337.856 |
| Heap | Ters sıralı | 100.000 | 269,05 | 2.926.640 | 4.492.302 |
| Selection | Kısmen sıralı | 100.000 | 229.325,0 | 4.999.950.000 | 29.997 |
| Insertion | Kısmen sıralı | 100.000 | 29.065,9 | 325.263.665 | 325.363.664 |
| Bubble | Kısmen sıralı | 100.000 | 273.544,0 | 4.999.578.750 | 975.490.999 |
| Shell | Kısmen sıralı | 100.000 | 359,63 | 3.465.588 | 4.972.731 |
| Quick | Kısmen sıralı | 100.000 | 102,02 | 1.626.512 | 159.214 |
| Merge | Kısmen sıralı | 100.000 | 234,31 | 1.419.708 | 3.337.856 |
| Heap | Kısmen sıralı | 100.000 | 292,50 | 3.103.405 | 4.926.951 |

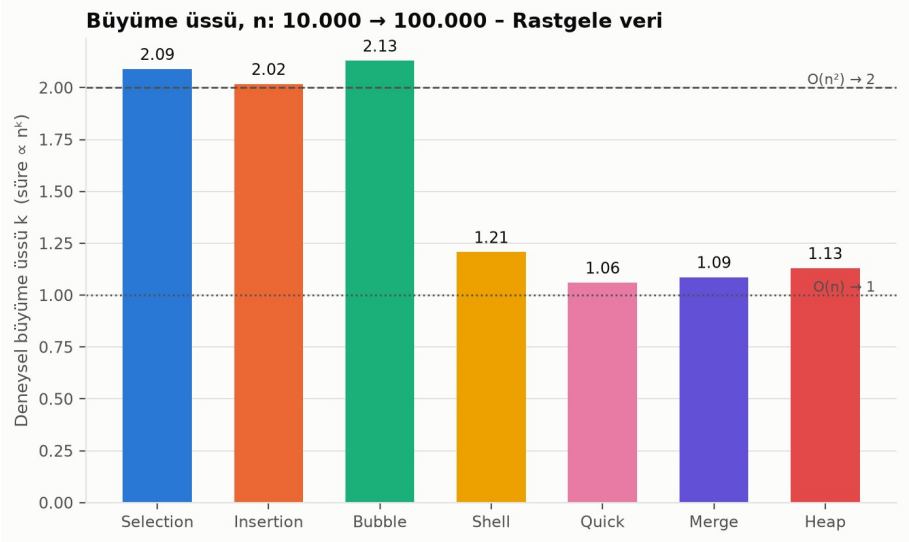
# 6. Karşılaştırma ve Tartışma

Aşağıda süre farklarını karşılaştırma, hareket ve veri düzeniyle birlikte değerlendiriyorum. Büyüme oranları aynı dağılımın farklı boyutlardaki ölçümlerinden hesaplanmıştır.

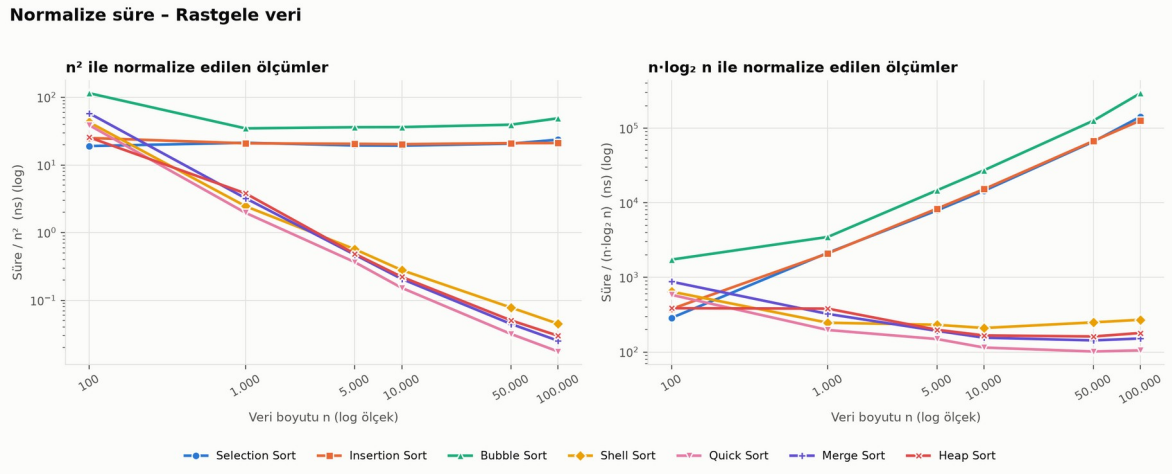
### 6.1 O($n^2$) algoritmaların süresi veri boyutu arttıkça nasıl değişti?

Rastgele veride n 1.000'den 100.000'e (100 kat) çıkınca Selection Sort'un süresi 11.128 kat, Insertion Sort'un 10.096 kat, Bubble Sort'un 14.097 kat arttı. Karesel modelde n 100 kat arttığında yaklaşık $100^2 = 10.000$ kat artış beklenir; oranlar bu referansla karşılaştırılmalıdır. n = 10.000'den 100.000'e deneysel büyüme üssü (süre $\propto n^k$) Selection için 2,09, Insertion için 2,02, Bubble için 2,13 bulundu (aşağıdaki büyüme üssü grafiği). Karşılaştırma sayıları teoriyle tam örtüşür: Selection Sort her dağılımda tam olarak $n(n-1)/2 = 4.999.950.000$

karşılaştırma yapar. n = 100.000'de Selection Sort yaklaşık 3,9 dk, Bubble Sort ise 8,2 dk sürmüştür. Bubble Sort'un diğer ikisinden yavaş olmasının nedeni, çok sayıda takas yapmasıdır: rastgele veride Bubble 7.515.812.496 veri hareketi yaparken Selection yalnızca 299.966 yapar.




*Şekil 32. Rastgele veride deneysel büyüme üssü (n: 10.000 → 100.000).*




*Şekil 33. Normalize süre: süre/n² ve süre/(n·log$_2$ n), rastgele veri.*

## 6.2 O(n log n) algoritmalar neden büyük veri kümelerinde avantajlıdır?

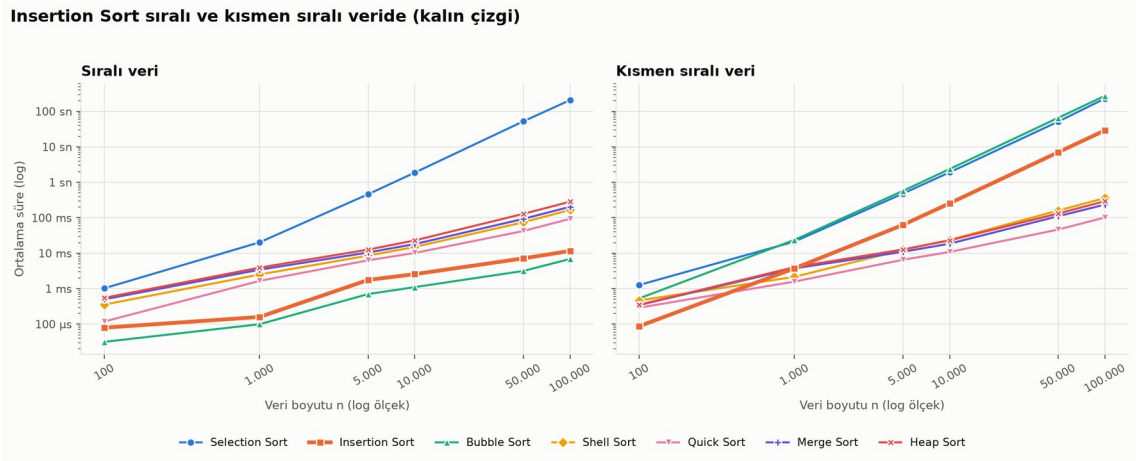
100.000 elemanlı rastgele girdide Quick Sort 174,8 ms sürer. Selection/Quick süre oranı 1.354, Bubble/Quick oranı 2.800'dür. Selection'ın her başlangıç konumunda kalan bölümün tamamını taraması, n(n−1)/2 karşılaştırmayı zorunlu kılar. Quick bu girdilerde bölümleri dengeli biçimde küçülterek daha az veri işlemi yapar.

Şekil 33 süreyi $n^2$ ve $n \cdot \log_2 n$ ile normalize eder. 100.000 elemanda Merge'in 1.536.353 karşılaştırması, yaklaşık 5 milyar karşılaştırma yapan Selection'dan çok düşüktür. Normalize eğrilerin ölçülen aralıkta yataylaşması ilgili büyüme modeliyle uyumludur. Shell aynı grafikte gösterilse de klasik gap için en kötü durum üst sınırı $O(n^2)$'dir.

### 6.3 Insertion Sort sıralı verilerde neden başarılıdır?

Insertion'ın kaydırma sayısı girdideki ters çift sayısına eşittir. Sıralı girdide sol komşuyla yapılan ilk karşılaştırma döngüyü durdurur; buna rağmen geçiciye alma ve son yazma yapılır. 100.000 elemanda bu nedenle 99.999 karşılaştırma ve 199.998 hareket kaydedilmiş, süre 11,6 ms olmuştur. Aynı veride erken duran Bubble 6,9 ms sürmüş ve hiç hareket yapmamıştır.

Şekil 34'te tam sıralı ve yaklaşık %10 konumu değiştirilmiş veriyi ayırır. İkinci durumda uzak konumların değişmesi çok sayıda ters çift oluşturabilir: 100.000 elemanda Insertion 325.263.665 karşılaştırma yapmış ve 29,1 sn sürmüştür. Bu, elemanların çoğunun başlangıç konumunda kalmasının tek başına doğrusal süreyi garanti etmediğini gösterir. Ters sıralı girdide her yeni eleman sol bölümün tamamını kaydırır; maliyet $O(n^2)$'ye ulaşır.




*Şekil 34. Insertion Sort sıralı ve kısmen sıralı veride (kalın çizgi).*

### 6.4 Quick Sort'un performansı pivot seçimine neden bağlıdır?

10.000 elemanlı sıralı girdide ilk pivot 50.014.998, orta pivot 125.439 karşılaştırma yapmıştır. İlk/orta süre oranı sıralı girdide 293,6, ters sırada 283,4'tür. Bölüm 5.8'deki süre oranı grafiği bu farkı gösterir. Sıralı girdide ilk değer en küçük olduğu için bölümleme bir sonraki tura neredeyse bütün diziyi bırakır; toplam karşılaştırma sayısı karesel büyür. Orta indeks pivotu bu özel girdide dengeli bölünme sağlar.

Rastgele girdide aynı boyut için ilk/orta süre oranı 1,10'dur. Bu, ilk pivotun her girdide aynı dezavantajı yaratmadığını gösterir. Ancak sabit bir orta indeks de bütün girdiler için iyi pivot garantisi vermez. Kod, küçük parçayı özyinelemeyle ve büyük parçayı döngüyle işlediğinden kötü bölümleme $O(n^2)$ zaman üretebilirken gerçek Python çağrı yığını $O(\log n)$ ile sınırlı kalır. Bu iki maliyet ayrı değerlendirilmelidir.

### 6.5 Merge Sort veri dağılımından nasıl etkileniyor?

Merge Sort, _araligi_sirala içinde sınırları ortadan bölerek çalışır. Her birleştirme işlenen aralığı önce tampona, sonra ana diziye yazar; bu yüzden aynı boyuttaki hareket sayısı dağılımdan bağımsızdır. 100.000 elemanda dört dağılımda da 3.337.856 hareket kaydedilmiştir. Karşılaştırmalar ise yarılardan birinin ne zaman tükendiğine bağlıdır: sıralı girdide 815.024, rastgele girdide 1.536.353 karşılaştırma vardır.

En yavaş dağılım süresinin en hızlıya oranı Merge için 1,24, Heap için 1,11, Selection için 1,14 bulundu. Bu kayıtlar Merge'in dağılıma az duyarlı olduğunu gösterse de en düşük oran her zaman Merge'e ait değildir. Süre değişkenliği ile eşit anahtarların sırasının korunması anlamındaki kararlılık ayrı özelliklerdir. Merge'in kararlılığı, _birlestir içindeki <= koşulunda eşitlikte soldaki elemanın seçilmesinden gelir.

### 6.6 Heap Sort ile Merge Sort büyük veri kümelerinde nasıl karşılaştırılır?

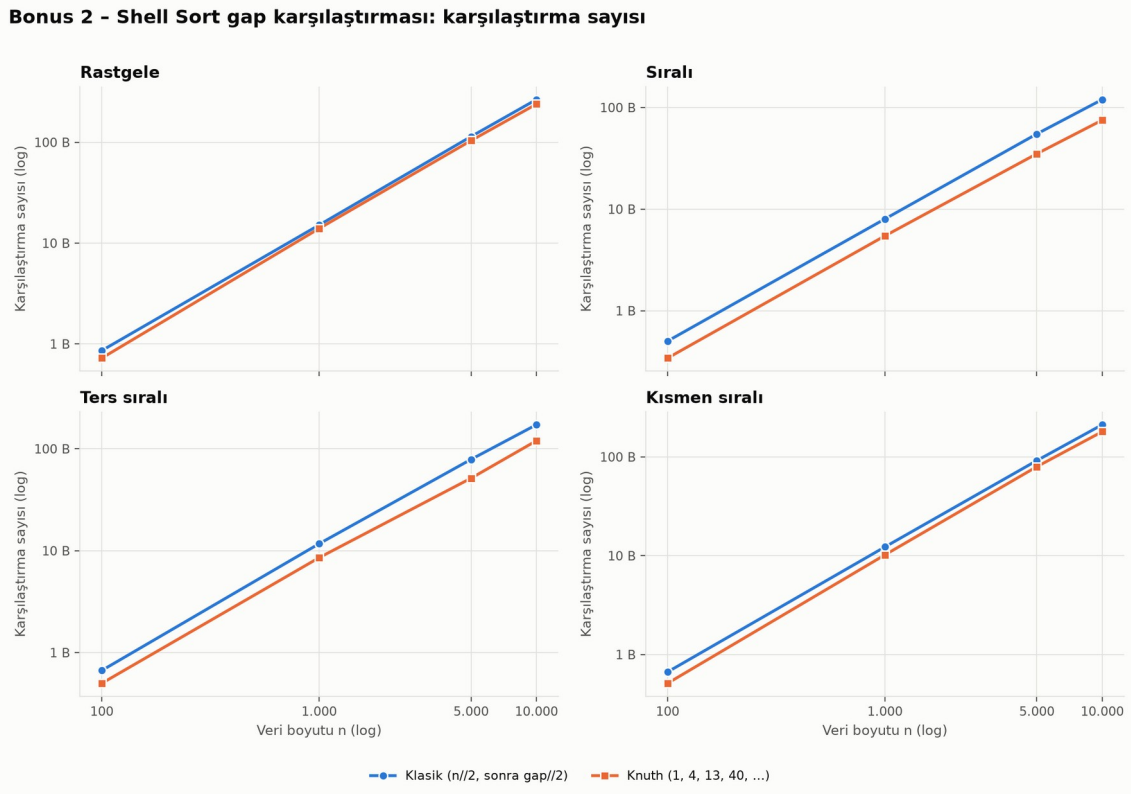
Rastgele 100.000 elemanda Merge 252,0 ms, Heap 298,6 ms sürer; Heap/Merge süre oranı 1,18'dir. 500.000 elemanda bu oran 1,20 olur. Bunlar iki ayrı boyuttaki gerçek ortalamalardır; sabit bir evrensel hız oranı değildir.

100.000 elemanda Heap 3.019.566, Merge 1.536.353 karşılaştırma yapar. Hareket sayıları aynı sırayla 4.724.852 ve 3.337.856'dır. Heap'in _asagi_suz fonksiyonu çocukları tek tek karşılaştırıp gerektiğinde üç hareketlik takas yapar; Merge ise sıralı yarıları tek geçişte tampona aktarır. Sayaç farkları süre farkını açıklamaya yardımcı olur, ancak her Python işleminin aynı maliyette olduğunu göstermez.

Bellek deneyinde 10.000 eleman için Merge 80.740, Heap 324 bayt tepe Python tahsisi üretmiştir. Kodda Merge'in n elemanlı tamponu, Heap'in ise yinelemeli ve tamponsuz süzmesi bulunur. Bu fark $O(n)$ ve $O(1)$ yardımcı alan açıklamalarıyla uyumludur. İşlemci önbelleği ayrıca ölçülmediğinden süre farkını kesin olarak önbelleğe bağlamıyorum.

### 6.7 Shell Sort'un performansı gap dizisinden nasıl etkilenir?

Gap aralıkları, bir sonraki ekleme geçişine kalan işi değiştirir. Şekil 35'te n=10.000 için rastgele veri karşılaştırma sayısı klasik yöntemde 265.793, Knuth yönteminde 239.323'tür. Ortalama süreler aynı sırayla 27,79 ve 23,82 ms'dir. İki yöntem aynı kaydırma kodunu kullanır; fark gap listesinden gelir. Gap üretimi her iki algoritma çağrısının içinde olduğundan ölçülen süreye dahildir. Klasik yöntemin rastgele veride 10.000–100.000 aralığındaki büyüme üssü 1,21 bulundu. Bu değer yalnızca iki ölçüm boyutuna ait ampirik eğimdir; O(n log n) garantisi veya tüm girdiler için bir zaman karmaşıklığı sonucu değildir. Klasik yöntemin $O(n^2)$ en kötü durum sınırı geçerliliğini korur (Knuth, 1998).




*Şekil 35. Klasik ve Knuth gap dizisi için karşılaştırma sayısı.*

### 6.8 Teorik Big-O değerleri deneysel sonuçlarla örtüşüyor mu?

**Tablo 10. Deneysel büyüme üssü k (süre ∝ $n^k$), n: 10.000 → 100.000**

| Algoritma | Rastgele | Sıralı | Ters sıralı | Kısmen sıralı |
|---|---|---|---|---|
| **Selection** | 2,09 | 2,04 | 2,06 | 2,08 |
| **Insertion** | 2,02 | 0,65 | 2,07 | 2,06 |
| **Bubble** | 2,13 | 0,79 | 2,06 | 2,06 |
| **Shell** | 1,21 | 1,04 | 1,10 | 1,18 |
| **Quick** | 1,06 | 0,96 | 0,95 | 0,97 |
| **Merge** | 1,09 | 1,05 | 1,05 | 1,10 |
| **Heap** | 1,13 | 1,09 | 1,10 | 1,10 |

Tablodaki üsler 10.000 ve 100.000 eleman için iki ortalama süreden hesaplanmıştır. Selection, Insertion ve Bubble'ın rastgele verideki üslerini 2 ile; Quick, Merge ve Heap'in değerlerini ise bu aralıkta n·log n modelinin verdiği yaklaşık 1,10 ile karşılaştırdım. Bu karşılaştırma yalnızca ölçülen aralığı açıklar. Algoritmanın bütün girdilerdeki davranışını kanıtlamak için süre eğrisi değil, kodun yaptığı işlemlerin analizi gerekir.

Kod ile kayıt arasında daha doğrudan bir kontrol Selection'da yapılabilir: 100.000 elemanda 4.999.950.000 karşılaştırma, $n(n-1)/2$ ifadesiyle tam olarak aynıdır. Merge'in her birleştirmede tampona ve ana diziye yazması ise aynı boyutta dağılımdan bağımsız hareket sayısı üretir. Shell'in eğimi gap listesinden ve veri düzeninden etkilenir; tabloda n log n modeline yakın olması klasik yöntemin $O(n^2)$ üst sınırını değiştirmez.

- Insertion'ın sıralı girdideki üssü 0,65 bulundu. Kodun $n-1$ karşılaştırma yaptığı kesindir; süre üssünün 1'den sapması ise farklı oturumlar, sabit maliyetler veya sistem gürültüsüyle ilişkili olabilir. Bu olasılıkların hangisinin etkili olduğunu belirleyecek ayrı bir ölçüm yapılmadı.

- Bellek tablosundaki bayt değerleri belirli Python tahsisleridir. Shell'in gap listesi ve Quick'in çağrı yığını için alan sınırı koddan O(log n) olarak çıkarılır; iki boyutta yakın tahsis değerleri ölçülmesi bu sınırı O(1)'e dönüştürmez.

### 6.9 Temel araştırma sorusunun yanıtı

Algoritma tercihini boyut, dağılım ve ek alan gereksinimi birlikte belirler. 100.000 elemanlı rastgele veride bu kayıtların en hızlısı orta pivotlu Quick Sort'tur. Sıralı girdide ise Insertion ve erken duran Bubble yalnızca $n-1$ veri karşılaştırması yapar. Yaklaşık %10 konumu

değiştirilen girdi, tam sıralı girdinin yerini tutmaz: uzak yer değiştirmeler çok sayıda kaydırma gerektirebilir.

En kötü durumda O(n log n) zaman sınırı isteniyorsa Merge ve Heap bu uygulamalar içinde uygun seçeneklerdir. Merge ek tampon karşılığında kararlı sıralama sağlar; Heap yinelemeli süzme sayesinde sabit ek alan kullanır. Shell'in klasik ve Knuth sonuçları gap seçiminin önemini gösterir, ancak ölçülen hız avantajı bütün veri kümelerine genellenemez.

## 7. Sonuç

Zorunlu deney matrisindeki 840 ölçümün tamamında sıralılık ve eleman bütünlüğü sağlandı. Deneyler, büyük rastgele girdilerde yapılan veri işlemlerinin süre farkını açıklamada yararlı olduğunu; sıralı veride ise erken durma ve kaydırma ihtiyacının sonucu değiştirdiğini gösterdi.

100.000 elemanlı rastgele veride Quick Sort 174,8 ms, Merge Sort 252,0 ms, Heap Sort 298,6 ms sürdü. İlk pivotun sıralı girdide orta pivota göre süre oranı, 10.000 elemanda 293,6 olarak ölçüldü. Bu fark çağrı yığınının büyümesinden değil, dengesiz bölünmelerde artan veri karşılaştırmalarından kaynaklanır. Shell için gözlenen eğim gap seçimine bağlıdır ve klasik yöntemin $O(n^2)$ üst sınırıyla karıştırılmamalıdır.

Sonuçlar bu Python gerçeklemeleri ve kayıtlı deney koşullarıyla sınırlıdır. Donanım sıcaklığı izlenmediği için süre farklarını ısınmaya bağlayamam. Paralel Merge için verilen sayılar model tahminidir; gerçek paralel hızlanma ölçülmemiştir. Bu ayrımlar, hangi sonuçların doğrudan ölçüme dayandığını ve hangilerinin ayrıca sınanması gerektiğini belirler.

## 8. Kaynakça

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley.

Hoare, C. A. R. (1962). Quicksort. The Computer Journal, 5(1), 10–16.

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference, 483–485.

Python Software Foundation. Python 3.13 documentation: time.perf_counter_ns, tracemalloc, threading. https://docs.python.org/3.13/library/time.html ; https://docs.python.org/3.13/library/tracemalloc.html ; https://docs.python.org/3.13/library/threading.html (erişim: 23.10.2026).

Sedgewick, R., & Wayne, K. Shell sınıfı belgeleri. https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Shell.html (erişim: 23.10.2026).

**Proje Kaynak Koduna Erişim**

Kaynak kod, otomatik testler, kayıtlı deney sonuçları ve README için proje deposu: https://github.com/Gulced/AlgoritmaSiralamaKarsilastirma